

A Comparative Study of Nonlinear MPC and Differential-Flatness-Based Control for Quadrotor Agile Flight

Sihao Sun, Angel Romero, Philipp Foehn, Elia Kaufmann and Davide Scaramuzza

Abstract—Accurate trajectory-tracking control for quadrotors is essential for safe navigation in cluttered environments. However, this is challenging in agile flights due to nonlinear dynamics, complex aerodynamic effects, and actuation constraints. In this article, we empirically compare two state-of-the-art control frameworks: the nonlinear-model-predictive controller (NMPC) and the differential-flatness-based controller (DFBC), by tracking a wide variety of agile trajectories at speeds up to 20 m/s (i.e., 72 km/h). The comparisons are performed in both simulation and real-world environments to systematically evaluate both methods from the aspect of tracking accuracy, robustness, and computational efficiency. We show the superiority of NMPC in tracking dynamically infeasible trajectories, at the cost of higher computation time and risk of numerical convergence issues. For both methods, we also quantitatively study the effect of adding an inner-loop controller using the incremental nonlinear dynamic inversion (INDI) method, and the effect of adding an aerodynamic drag model. Our real-world experiments, performed in one of the world's largest motion capture systems, demonstrate more than 78% tracking error reduction of both NMPC and DFBC, indicating the necessity of using an inner-loop controller and aerodynamic drag model for agile trajectory tracking.

MULTIMEDIA MATERIAL

The experimental results can be viewed in this video:
https://youtu.be/XpuRpKHp_Bk

I. INTRODUCTION

A. Motivation

Quadrotors are extremely agile. Exploiting their agility is crucial for time-critical missions, such as search and rescue, monitoring, exploration, aerial delivery, drone racing, reconnaissance, and even flying cars [1, 2]. An accurate trajectory-tracking controller is required to safely execute high-speed trajectories in cluttered environments. However, most approaches struggle to handle joint effects in agile flights, such as nonlinear dynamics, aerodynamic effects, and actuation limits.

Recently, nonlinear model predictive control (NMPC) has drawn much attention for quadrotor control thanks to advances

All authors are with the Robotics and Perception Group, University of Zurich, Switzerland (<http://rpg.ifi.uzh.ch>). This work was supported by in part by the National Centre of Competence in Research (NCCR) Robotics, through the Swiss National Science Foundation (SNSF), in part by the European Union's Horizon 2020 Research and Innovation Programme under grant agreement No. 871479 (AERIAL-CORE), and in part by the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT).

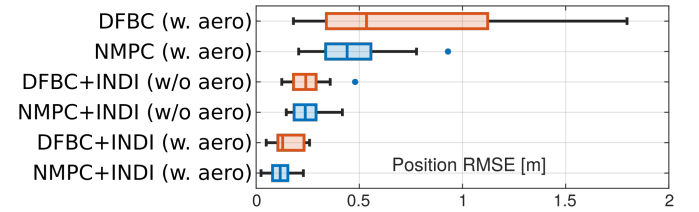
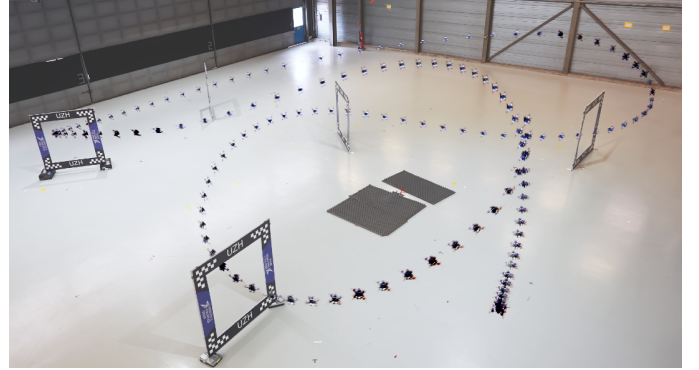


Fig. 1: **Top:** Our quadrotor tracking a race trajectory. **Bottom:** Box plot comparing the position tracking root-mean-square-error (RMSE) of NMPC, DFBC, and their variations with INDI inner-loop, with or without considering aerodynamic drag effects. For each method, data are collected from real-world flights tracking different reference trajectories with speeds up to 20 m/s (i.e., 72 km/h) and accelerations up to 5g.

in hardware and algorithmic efficiency [3–11]. NMPC particularly excels in handling control limits, and its predictive nature is believed to be beneficial for trajectory tracking at high speeds [3]. A recent study has demonstrated its performance in tracking aggressive trajectories up to 20 m/s [4].

However, NMPC is computationally extremely demanding compared to the state-of-the-art non-predictive method: the differential-flatness-based controller (DFBC) [12, 13]. This method also shows great potential in accurately tracking agile trajectories autonomously. A recent study has used DFBC to track trajectories up to 12.9 m/s with 2.1 g accelerations, with only 6.6 centimeters position tracking error [13]. Although the state-of-the-art DFBC has not achieved agile flights as fast as NMPC showed in [4], its high computational efficiency and tracking accuracy render the necessity of applying NMPC for agile trajectory tracking questionable. Therefore, a comparative study of NMPC and DFBC is needed to provide insights

四旋翼敏捷飞行的非线性MPC与基于微分平坦性的控制的比较研究

孙思浩, 安吉尔·罗梅罗, 菲利普·费恩, 埃利亚·考夫曼 和 达维德·斯卡拉穆扎

Abstract—对四旋翼无人机进行精确轨迹跟踪控制对于在复杂环境中安全导航至关重要。然而, 在灵活飞行中, 由于非线性动力学、复杂的空气动力学效应和执行约束, 这一任务具有挑战性。在本文中, 我们通过跟踪速度高达 20 m/s (即 72 km/h) 的各种灵活轨迹, 从实验角度比较了两种最先进的控制框架: 非线性模型预测控制器 (NMPC) 和基于微分平坦性的控制器 (DFBC)。这些比较在仿真和实际环境中进行, 以系统评估两种方法在跟踪精度、鲁棒性和计算效率方面的表现。我们展示了NMPC在跟踪动态不可行轨迹方面的优势, 但代价是更高的计算时间和数值收敛问题的风险。对于两种方法, 我们还定量研究了通过增添内环控制器所产生的影响。

多媒体材料

实验结果可以在此视频中观看: <https://youtu.be/XpuRpKHpbK>

一、引言

A. Motivation

四旋翼无人机极其灵活。利用其灵活性对于时间紧迫的任务至关重要, 例如搜索与救援、监控、探索、空中投送、无人机竞速、侦察, 甚至飞行汽车 [1, 2]。要在复杂环境中安全执行高速轨迹, 需要一个精确的轨迹跟踪控制器。然而, 大多数方法在处理灵活飞行中的联合效应时存在困难, 例如非线性动力学、空气动力学影响和驱动限制。

最近, 由于技术进步, 非线性模型预测控制 (NMPC) 因四旋翼控制而受到广泛关注

All authors are with the Robotics and Perception Group, University of Zurich, Switzerland (<http://rpg.ifi.uzh.ch>). This work was supported by in part by the National Centre of Competence in Research (NCCR) Robotics, through the Swiss National Science Foundation (SNSF), in part by the European Union's Horizon 2020 Research and Innovation Programme under grant agreement No. 871479 (AERIAL-CORE), and in part by the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT).

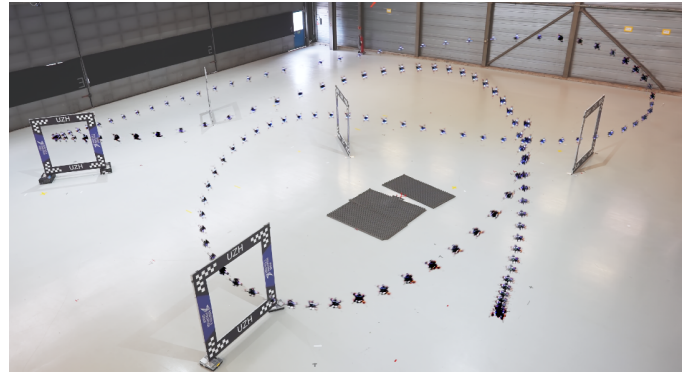


图1: 上: 我们的四旋翼无人机跟踪赛道轨迹。下: 箱线图比较了NMPC、DFBC及其带有INDI内环的变体的位置信息跟踪均方根误差 (RMSE), 并区分是否考虑空气动力阻力效应。对于每种方法, 数据均来自实际飞行, 跟踪不同参考轨迹, 其速度高达 20 m/s (即72 km/h), 加速度高达5g。

在硬件和算法效率方面 [3–11]。NMPC 在处理控制极限方面尤其出色, 其预测特性被认为对高速下的轨迹跟踪有益 [3]。一项最新研究已展示了其在跟踪速度高达 20 m/s 的激进轨迹方面的性能 [4]。

然而, 与最先进的非预测方法——基于微分平坦性的控制器 (DFBC) [12, 13]相比, NMPC在计算上极为耗费资源。该方法在自主精确跟踪灵巧轨迹方面也显示出巨大潜力。最近的一项研究使用DFBC跟踪速度达到12.9 m/s、加速度为2.1 g的轨迹, 位置跟踪误差仅为6.6厘米 [13]。尽管最先进的DFBC尚未实现像文献[4]中NMPC所展示的那样快速的灵巧飞行, 但其高计算效率和跟踪精度使得是否有必要将NMPC应用于灵巧轨迹跟踪成为疑问。因此, 需要对NMPC和DFBC进行比较研究以提供见解。

for future research on fully autonomous agile flights, in order to further improve their efficiency and reliability.

B. Contribution

This article presents the first comparative study of the two state-of-the-art tracking controllers: NMPC and DFBC method, in fast trajectory tracking with speed up to 20 m/s (i.e., 72 km/h) and 5 g accelerations. Specifically, we select the NMPC method that uses the full nonlinear dynamics with proper actuator bounds and regards single rotor thrusts as control inputs. This NMPC has been applied in previous works, such as [3, 5]. For a fair comparison, we improve the DFBC method used in [12, 13] with a control allocation approach using constrained quadratic programming [14] also to consider control input limits.

All experiments are conducted in both simulations and the real world. The simulations compare in both ideal and practical conditions with model-mismatch, estimation latency, and external disturbances. The real-world experiments are conducted in one of the world's largest motion capture systems, with $30 \times 30 \times 8$ m³ flight volume. Multiple agile trajectories are selected as reference, including not only dynamically *feasible* trajectories, but also dynamically *infeasible* trajectories that require thrusts exceeding the maximum capacity of the quadrotor motors, which is likely to happen in high-speed flights due to model mismatch. These tests investigate the performance of both methods in the presence of significant high-speed-induced aerodynamic effects, inevitable system latency, and on an onboard embedded computer.

The experimental results reveal that NMPC is considerably more computationally demanding, and more prone to suffer from numerical convergence issues in the presence of large external force disturbances. However, NMPC also excels in tracking dynamically *infeasible* trajectories, making it more suitable for tracking time-optimal trajectories that violate the rotor thrust constraints.

In addition, this study also highlights the importance of implementing an inner-loop controller for robustification. Specifically, we select the incremental nonlinear dynamic inversion (INDI) method as the inner-loop angular controller, thanks to its simplicity in implementation and demonstrated robustness in various real-world experiments [15, 16] including a combination with DFBC [13]. As for NMPC, differently from the state-of-the-art using a PID as the low-level control [4], we propose a method to hybridize NMPC with INDI that considers the real input limits of the quadrotor instead of constraints on virtual inputs. Real-world flight results demonstrate more than 78% position tracking error reduction of NMPC and DFBC with an INDI inner-loop. We also reveal that a well-selected inner-loop controller is more crucial than simply considering the aerodynamic effects, as is compared in Fig. 1.

Apart from the technical contribution, this paper can also be regarded as a tutorial for non-expert readers on agile quadrotor flight. We encourage the practitioners to use the presented results as a baseline for further development of both DFBC and NMPC approaches.

II. RELATED WORK

In this paper, we classify the trajectory tracking controller into two categories: non-predictive and predictive methods. While the predictive methods encode multiple future time-steps into the control command, the non-predictive methods only track a single reference step. In the following, we review works towards improving quadrotor trajectory tracking accuracy from these two different categories. A more comprehensive survey of quadrotor position and attitude control methods can be found in the literature (e.g., see [17, 18]).

A. Non-predictive Quadrotor Trajectory Tracking Control

Unlike most fixed-wing aircraft, quadrotors are inherently unstable. Therefore, the initial work of quadrotor control aimed at achieving stable hovering and near-hover flights. Thanks to the small-angle assumptions in these conditions, linear control methods such as PID and LQR demonstrate sufficiently good performance (see, e.g., [19, 20]).

However, as the requirements for agile flight emerges, these assumptions are no longer valid. Nonlinearities from the attitude dynamics are the first problem to tackle. For this reason, nonlinear flight controllers are proposed, such as feedback linearization [21] and backstepping [22]. In order to cope with the singularities of Euler angles as the nonlinear attitude representation, quaternions are widely adopted to parametrize the attitude [23]. In addition, the authors of [24] propose the geometric tracking controller to directly control the quadrotor on the manifold of the special Euclidean group SE(3), showing almost globally asymptotic tracking of the position, velocity, and attitude of the quadrotor.

A seminal work [25] reveals that quadrotors are differentially flat systems. By virtue of this property, given the time-parameterized 3D path, one can derive the reference attitude, angular rate, and accelerations. These references can be sent to a closed-loop flight controller as feedforward terms, while additional feedback control is required to address model mismatch and external disturbances. As such, differential-flatness based controller (DFBC) has significantly improved the tracking performance at relatively high speeds [12, 13].

As the flying speed increases, a quadrotor starts experiencing non-negligible aerodynamic effects, including drag [26], aerodynamic torque [27], and variation of thrusts [28]. Authors of [12] show that the aerodynamic drag does not affect the differential flatness of a quadrotor. Thus they adopt a first order aerodynamic model with feedforward terms derived from the reference trajectory to improve the tracking performance. Since an accelerometer can directly read external forces, [13] leverages accelerometer measurements to improve the tracking accuracy instead of resorting to an aerodynamic drag model. This method also demonstrates remarkable performance in disturbance rejection and platform adaptability.

Effectively handling control input limits is a remaining challenge for non-predictive methods, including DFBC. So far, existing methods have prioritized the position tracking over heading using various approaches, such as redistributed pseudo inversion [29], weighted-least-square allocation [30], control-prioritization method [31, 13], and constrained-quadratic-

为了未来对全自主灵活飞行的研究，以进一步提高其效率和可靠性。

B. Contribution

本文首次对两种最先进的跟踪控制器——NMPC 和 DFBC 方法在速度达 20 m/s (即 72 km/h) 和 5 g 加速度下的快速轨迹跟踪性能进行了对比研究。具体而言，我们选择了使用全非线性动力学、具有适当执行器边界并将单个转子推力视为控制输入的 NMPC 方法。该 NMPC 方法已在先前的研究中应用，如 [3, 5]。为了公平比较，我们改进了 [12, 13] 中使用的 DFBC 方法，通过采用受约束的二次规划 [14] 的控制分配方法，也考虑了控制输入的限制。

所有实验都在仿真和现实环境中进行。仿真在理想和实际条件下进行比较，包括模型不匹配、估计延迟以及外部干扰。现实世界的实验在世界上最大的运动捕捉系统之一进行，飞行空间为 30×30×8 米。选择了多条高机动性轨迹作为参考，包括不仅是动态 *feasible* 轨迹，还有动态 *infeasible* 轨迹，这些轨迹需要的推力超过四旋翼电机的最大能力，这在高速飞行中由于模型不匹配可能会发生。这些测试研究了两种方法在存在显著高速引起的气动效应、不可避免的系统延迟以及机载嵌入式计算机条件下的性能。

实验结果表明，NMPC 的计算需求显著更高，并且在存在较大外力干扰的情况下更容易出现数值收敛问题。然而，NMPC 在跟踪动态 *infeasible* 轨迹方面也表现出色，使其更适合跟踪违反转子推力约束的时间最优轨迹。

此外，本研究还强调了实施内环控制器以提高稳健性的重要性。具体来说，我们选择增量非线性动态反演 (INDI) 方法作为内环角速度控制器，因为它在实现上简单，并且在包括与 DFBC 结合的多种实际实验中表现出稳健性 [15, 16, 13]。至于 NMPC，与使用 PID 作为低级控制器的现有方法不同 [4]，我们提出了一种将 NMPC 与 INDI 混合的方法，该方法考虑了四旋翼的实际输入限制，而不是虚拟输入的约束。实际飞行结果表明，NMPC 和 DFBC 结合 INDI 内环的情况下，位置跟踪误差降低了超过 78%。我们还发现，精心选择的内环控制器比单纯考虑气动效应更加关键，如图 1 所示的对比所示。

除了技术贡献外，本文还可以被视为面向非专业读者的敏捷四旋翼飞行教程。我们鼓励从业者将所呈现的结果作为进一步开发 DFBC 和 NMPC 方法的基准。

二、相关工作

在本文中，我们将轨迹跟踪控制器分为两类：非预测方法和预测方法。预测方法将多个未来时间步的信息编码到控制指令中，而非预测方法仅跟踪单个参考步骤。接下来，我们将从这两类方法的角度回顾提高四旋翼轨迹跟踪精度的相关工作。有关四旋翼位置和姿态控制方法的更全面综述，可参见文献（例如，见 [17, 18]）。

A. Non-predictive Quadrotor Trajectory Tracking Control

与大多数固定翼飞机不同，四旋翼本质上是不稳定的。因此，四旋翼控制的初步工作旨在实现稳定的悬停和近悬停飞行。得益于这些条件下的小角度假设，线性控制方法如 PID 和 LQR 展现出足够良好的性能（参见，例如，[19, 20]）。

然而，随着对敏捷飞行的需求出现，这些假设不再成立。姿态动力学的非线性是首先需要解决的问题。为此，提出了非线性飞行控制器，例如反馈线性化 [21] 和回步法 [22]。为了应对作为非线性姿态表示的欧拉角奇异性，四元数被广泛采用来参数化姿态 [23]。此外，[24] 的作者提出了几何跟踪控制器，直接在特殊欧几里得群 $SE(3)$ 的流形上控制四旋翼，显示出对四旋翼的位置、速度和姿态几乎全局渐近跟踪的能力。

一项开创性工作 [25] 表明，四旋翼是差分平坦系统。凭借这一特性，给定时间参数化的三维路径后，可以推导出参考姿态、角速度和加速度。这些参考值可以作为前馈项发送给闭环飞行控制器，同时需要额外的反馈控制来解决模型不匹配和外部干扰。因此，基于差分平坦性的控制器 (DFBC) 在相对高速下显著提高了跟踪性能 [12, 13]。

随着飞行速度的增加，四旋翼开始经历不可忽视的空气动力学效应，包括阻力 [26]、气动扭矩 [27] 和推力变化 [28]。《[12]》的作者表明，空气动力阻力不会影响四旋翼的微分平坦性。因此，他们采用了一阶空气动力学模型，并从参考轨迹中导出的前馈项以提高跟踪性能。由于加速度计可以直接测量外部力，《[13]》利用加速度计测量来提高跟踪精度，而不是依赖空气动力阻力模型。该方法在抗扰能力和平台适应性方面也表现出了显著性能。

有效处理控制输入限制仍然是包括 DFBC 在内的非预测方法面临的一个挑战。到目前为止，现有方法已经使用各种方法优先考虑位置跟踪而非航向控制，例如重新分配的伪逆 [29]、加权最小二乘分配 [30]、控制优先方法 [31, 13] 以及约束二次-

programming allocation [32]. While these methods can mitigate the actuator saturation effect when the trajectories are dynamically *feasible*, its performance in tracking dynamically *infeasible* trajectories is still questionable. In this work, we will push the limit of DFBC to conduct agile trajectory tracking tasks to a much higher flight speed and acceleration than the state-of-the-art, and study its performance in tracking dynamically *infeasible* trajectories where violating rotor thrust limits becomes inevitable.

B. Model Predictive Control for Quadrotor Trajectory Tracking

Model predictive control (MPC) is a prevalent method in robots thanks to its predictive nature and ability to handle input constraints [10, 33]. It generates control commands in receding horizon fashion, which minimizes the tracking error in the predicted time horizon by solving constrained optimization problems.

However, MPC is very computationally demanding compared with non-predictive methods. Especially, using nonlinear-MPC (NMPC) with a full-state nonlinear model of a quadrotor was computationally intractable onboard early-age unmanned-aerial vehicles (UAVs). For this reason, linear-MPC (LMPC) is adopted in many studies for only position control [34], or control linearized model based on small-angle assumptions [35]. Therefore, these LMPC methods cannot fully capture nonlinearities in rotational dynamics, and underperforms NMPC methods [10, 33]

Separating flight controllers into high-level position control and low-level attitude control is another common approach to simplify the model and alleviate the computational load of NMPC [6, 7, 9]. However, in such a cascaded control architecture, the high-level controller cannot precisely capture the real quadrotor capability because they often ignore the effects of system limitation (such as [6]), or introduce a virtual constraint on states [9]. Consequently, the command fed into the lower-level controller is either too conservative to achieve agile flights, or over-aggressive that causes instability.

Thanks to the recent development in hardware and nonlinear optimization solvers [36–38], running NMPC with a nonlinear full dynamic model becomes computationally tractable on an embedded computer. Hence, recently, some studies start using the full-nonlinear dynamic model, and regarding single rotor thrusts as inputs for NMPC, thus are able to fully exploit quadrotors capability [3–5]. These methods either directly use the optimized single rotor thrust commands [3], or send intermediate states from the solution (such as the angular rates) to a low-level controller [4, 5]. A recent study [4] demonstrates the ability of the full-model NMPC with a PID low-level controller in tracking a pre-planned race trajectory at speed up to 20 m/s which surpasses the top speed of 12.9 m/s reported in [13] using DFBC, in spite of a much larger tracking error (0.7 m v.s. 0.066 m).

Although running NMPC is realizable on modern embedded computers, it still requires significantly more computational resources than non-predictive methods represented by DFBC. For this reason, NMPC may suffer from numerical convergence issues, especially when the platform lacks a sufficient

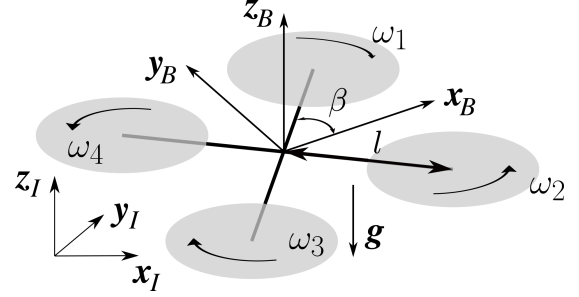


Fig. 2: Coordinate definitions and propeller numbering convention.

computational budget. Moreover, the advantage of NMPC becomes questionable as DFBC can also address input limits using the control allocation technique and generate feedforward control leveraging differential-flatness property. Therefore, it is necessary to compare these two methodologies and understand at what conditions each approach is preferable to provide insights and recommendations for future applications.

III. PRELIMINARIES

A. Notations

Throughout the paper, we use subscription r to indicate the reference variables derived from the reference trajectory. Subscription d indicates the desired value, which is calculated from a higher-level controller. We use bold lowercase letters to represent vectors and bold uppercase letters for matrices; otherwise, they are scalars. Two right-handed coordinate frames are used in this paper: they are the inertial-frame $\mathcal{F}_I : \{\mathbf{x}_I, \mathbf{y}_I, \mathbf{z}_I\}$ with $\mathbf{z}_I$ pointing upward opposite to the gravity, and the body-frame $\mathcal{F}_B : \{\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B\}$ with $\mathbf{x}_B$ pointing forward and $\mathbf{z}_B$ aligned with the collective thrust direction (see Fig. 2). Vectors with superscription B are expressed in the body-frame; others without any superscription are expressed in the inertial-frame. The rotation from $\mathcal{F}_I$ to $\mathcal{F}_B$ is represented by rotational matrix $\mathbf{R}(\mathbf{q}) = [\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B] \in \text{SO}(3)$ parameterized by quaternion $\mathbf{q} = [q_w, q_x, q_y, q_z]^T \in \mathbb{S}^3$. We use subscript $\{x, y, z\}$ to represent the imaginary components of a quaternion, namely $\mathbf{q}_{x,y,z} = [q_x, q_y, q_z]^T$. Operator $\text{diag}(A_1, A_2, \dots, A_n)$ denotes a diagonal matrix with scalars or matrices $(A_1, A_2, \dots, A_n)$ as diagonal entries.

B. Quadrotor Model

1) *Quadrotor Rigid-Body Model*: The quadrotor model is established using 6-DoF rigid body kinematic and dynamic equations. For translational dynamics, we have

$$\ddot{\boldsymbol{\xi}} = (T\mathbf{z}_B + \mathbf{f}_a)/m + \mathbf{g}, \quad (1)$$

where $\boldsymbol{\xi}$ denotes the position of the quadrotor center of gravity (CoG); T and m are the collective thrust and total mass respectively; $\mathbf{g} \in \mathbb{R}^3$ is the gravitational vector; $\mathbf{f}_a$ indicates the exogenous aerodynamic drag force during high-speed flights.

The rotational kinematic and dynamic equations are expressed as

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\Omega}^B \end{bmatrix}, \quad (2)$$

编程分配 [32]。虽然这些方法可以在轨迹动态为 *feasible* 时减轻执行器饱和效应，但在跟踪动态为 *infeasible* 的轨迹时，其性能仍然存在疑问。在本工作中，我们将推动 DFBC 的极限，以在比最先进技术更高的飞行速度和加速度下执行敏捷轨迹跟踪任务，并研究其在跟踪动态为 *infeasible* 的轨迹时的性能，在该情况下违反转子推力极限是不可避免的。

B. Model Predictive Control for Quadrotor Trajectory Tracking

模型预测控制 (MPC) 由于其预测特性和处理输入约束的能力，在机器人中是一种流行的方法 [10, 33]。它以递推时域的方式生成控制指令，通过求解约束优化问题来最小化预测时间范围内的跟踪误差。

然而，与非预测方法相比，MPC 的计算需求非常高。尤其是，对于早期无人机 (UAV)，使用基于四旋翼全状态非线性模型的非线性 MPC (NMPC) 在机载上是计算上难以实现的。因此，许多研究中采用线性 MPC (LMPC) 仅用于位置控制 [34]，或用于基于小角度假设的线性化控制模型 [35]。因此，这些 LMPC 方法无法完全捕捉旋转动力学中的非线性特性，其性能低于 NMPC 方法 [10, 33]

将飞行控制器分为高层位置控制和低层姿态控制是简化模型和减轻 NMPC 计算负荷的另一种常见方法 [6, 7, 9]。然而，在这种级联控制架构中，高层控制器无法准确捕捉实际四旋翼的能力，因为它们通常会忽略系统限制的影响 (例如 [6])，或在状态上引入虚拟约束 [9]。因此，传递到低层控制器的指令要么过于保守，无法实现灵活飞行，要么过于激进，从而导致不稳定。

得益于硬件和非线性优化求解器的最新发展 [36–38]，在嵌入式计算机上使用非线性全动态模型运行 NMPC 变得在计算上可行。因此，最近一些研究开始使用全非线性动态模型，并将单个旋翼推力作为 NMPC 的输入，从而能够充分利用四旋翼的能力 [3–5]。这些方法要么直接使用优化后的单旋翼推力命令 [3]，要么将解中的中间状态 (例如角速度) 发送给低级控制器 [4, 5]。一项最新研究 [4] 展示了采用 PID 低级控制器的全模型 NMPC 在以高达 20 m/s 的速度跟踪预先规划的竞赛轨迹时的能力，这超过了在 [13] 中使用 DFBC 报告的最高速度 12.9 m/s，尽管跟踪误差更大 (0.7 m 对比 0.066 m)。

虽然在现代嵌入式计算机上运行 NMPC 是可实现的，但它仍然比以 DFBC 为代表的非预测方法需要显著更多的计算资源。因此，NMPC 可能会遇到数值收敛问题，特别是当平台缺乏足够的

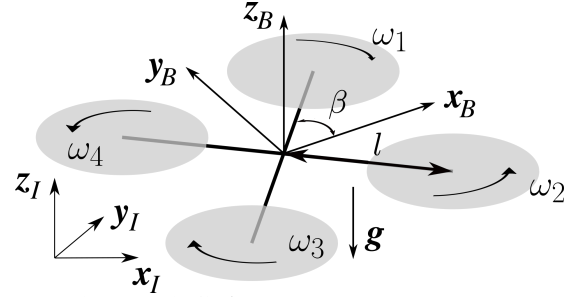


图 2: 坐标定义和螺旋桨编号规则。

计算预算。此外，NMPC 的优势变得值得怀疑，因为 DFBC 也可以使用控制分配技术来处理输入限制，并利用微分平坦性特性生成前馈控制。因此，有必要比较这两种方法，并了解在何种条件下每种方法更可取，以为未来的应用提供见解和建议。

III. 初步工作

A. Notations

在整篇论文中，我们使用下标 r 来表示从参考轨迹导出的参考变量。下标 d 表示期望值，它由更高层的控制器计算得出。我们使用加粗小写字母表示向量，加粗大写字母表示矩阵；否则，它们为标量。本文使用了两个右手坐标系：惯性坐标系 $\mathcal{F}_I: \{\mathbf{x}_I, \mathbf{y}_I, \mathbf{z}_I\}$ ，其中 $\mathbf{z}_I$ 指向与重力相反的上方；机体坐标系 $\mathcal{F}_B: \{\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B\}$ ，其中 $\mathbf{x}_B$ 指向前方， $\mathbf{z}_B$ 与总推力方向对齐 (见图 2)。带上标 B 的向量在机体坐标系中表示；其他未带上标的向量在惯性坐标系中表示。从 $\mathcal{F}_I$ 到 $\mathcal{F}_B$ 的旋转由旋转矩阵 $\mathbf{R}(\mathbf{q}) = [\mathbf{x}_B, \mathbf{y}_B, \mathbf{z}_B] \in \mathbb{R}^{3 \times 3}$ 表示，采用四元数 $\mathbf{q} = [q_w, q_x, q_y, q_z]^T \in \mathbb{S}^3$ 参数化。我们使用下标 $\{x, y, z\}$ 表示四元数的虚部，即 $\mathbf{q}_{x,y,z} =$ 。

B. Quadrotor Model

1) *Quadrotor Rigid-Body Model*: 四旋翼模型是使用六自由度刚体运动学和动力学方程建立的。对于平移动力学，我们有

$$\ddot{\boldsymbol{\xi}} = (T\mathbf{z}_B + \mathbf{f}_a)/m + \mathbf{g}, \quad (1)$$

其中 $\boldsymbol{\xi}$ 表示四旋翼重心 (CoG) 的位置； T 和 m 分别是总推力和总质量； $\mathbf{g} \in \mathbb{R}^3$ 是重力向量； $\mathbf{f}_a$ 表示高速飞行期间的外生空气阻力。

旋转运动学和动力学方程表示为

$$\dot{\mathbf{q}} = \frac{1}{2}\mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\Omega}^B \end{bmatrix}, \quad (2)$$

$$\mathbf{I}_v \dot{\boldsymbol{\Omega}}^B = \mathbf{I}_v \boldsymbol{\alpha}^B = -\boldsymbol{\Omega}^B \times \mathbf{I}_v \boldsymbol{\Omega}^B + \boldsymbol{\tau} + \mathbf{d}_\tau, \quad (3)$$

where $\otimes$ denotes the quaternion multiplication operator. $\boldsymbol{\Omega}$ is the angular velocity of $\mathcal{F}_B$ with respect to $\mathcal{F}_I$. Its derivative, namely angular acceleration, is denoted by $\boldsymbol{\alpha}$. Throughout the paper, we use its projection on $\mathcal{F}_B$, namely $\boldsymbol{\Omega}^B = [\Omega_x, \Omega_y, \Omega_z]^T$, since its directly measurable from the inertial measurement unit (IMU). $\mathbf{I}_v$ indicates the inertia matrix of the entire quadrotor. $\boldsymbol{\tau}$ is the resultant torque generated by rotors. $\mathbf{d}_\tau$ is the model uncertainties on the body torque, which can come from high-order aerodynamic effects, center of gravity bias, or distinction among rotors.

The collective thrust and rotor generated torques are functions of rotor speeds:

$$\begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix} = \mathbf{G}_1 \mathbf{u} + \mathbf{G}_2 \dot{\boldsymbol{\omega}} + \mathbf{G}_3 (\boldsymbol{\Omega}) \boldsymbol{\omega} \quad (4)$$

where

$$\mathbf{u} = c_t \boldsymbol{\omega}^{\circ} \quad (5)$$

represents the thrust generated by each rotor and $^\circ$ indicates the Hadamard power. c_t is the thrust coefficient. $\boldsymbol{\omega}$ is the vector of angular rates of each propeller. $\mathbf{G}_1$ to $\mathbf{G}_3$ are matrices defined as

$$\mathbf{G}_1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ l \sin \beta & -l \sin \beta & -l \sin \beta & l \sin \beta \\ -l \cos \beta & -l \cos \beta & l \cos \beta & l \cos \beta \\ c_q/c_t & -c_q/c_t & c_q/c_t & -c_q/c_t \end{bmatrix} \quad (6)$$

$$\mathbf{G}_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ I_p & -I_p & I_p & -I_p \end{bmatrix} \quad (7)$$

$$\mathbf{G}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ I_p \Omega_y & -I_p \Omega_y & I_p \Omega_y & -I_p \Omega_y \\ -I_p \Omega_x & I_p \Omega_x & -I_p \Omega_x & I_p \Omega_x \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (8)$$

where c_q is the torque coefficient; β and l are geometric parameters defined as per Fig. 2. I_p is the inertia of the rotor around z_B .

Terms $\mathbf{G}_2 \dot{\boldsymbol{\omega}}$ and $\mathbf{G}_3 (\boldsymbol{\Omega}) \boldsymbol{\omega}$ are torques respectively due to angular acceleration of rotors and gyroscopic effects, which are usually neglected for controller design. However, in Sec IV-C, we will revisit the INDI method [13] for angular acceleration control that takes into account effects of the inertial torque term $\mathbf{G}_2 \dot{\boldsymbol{\omega}}$.

2) *Aerodynamic Drag Model*: Quadrotors during high-speed flight experience significant aerodynamic drag forces, which need to be precisely modeled to improve tracking accuracy while minimizing the computational overhead. In this work, we use an aerodynamic drag model which captures the major effects, and is proved effective in works such as [12].

$$\mathbf{f}_a^B = \begin{bmatrix} -k_{d,x} v_x \\ -k_{d,y} v_y \\ -k_{d,z} v_z + k_h (v_x^2 + v_y^2) \end{bmatrix} \quad (9)$$

where $[v_x, v_y, v_z] = \mathbf{R}(\mathbf{q})^T \dot{\boldsymbol{\xi}}$ (i.e., the projection of velocity in the body frame; here we assume zero wind-speed). $k_{d,x,y,z}$ and k_h are positive parameters can be identified from flight data.

IV. METHODOLOGIES

This section elaborates the two control methods compared. A nonlinear NMPC method is selected that considers the thrust limits of each rotor, the full nonlinear dynamics, and the aerodynamic effects. As for the DFBC method, we improve the state-of-the-art such that these factors are also addressed, which ensures a fair comparison with NMPC. Finally, both methods are augmented with an INDI controller [15] to convert the single rotor thrust commands to rotor-speed commands, while improving the robustness against model uncertainties and disturbances on the rotational dynamics. The control diagrams of both methods are illustrated in Fig. 4 and Fig. 3.

A. Nonlinear Model Predictive Controller

NMPC generates control commands by solving a finite-time optimal control problem (OCP) in a receding horizon fashion. Given a reference trajectory, the cost function is the error between the predicted states and the reference states in the time horizon, meaning that multiple reference points in the time horizon are used. In order to conduct numerical optimizations, we discretize the states and inputs into N equal intervals over the time horizon $\tau \in [t, t+h]$ of size $dt = h/N$ with h denoting the horizon length, yielding a constrained nonlinear optimization problem:

$$\begin{aligned} \mathbf{u}_{\text{NMPC}} = \underset{\mathbf{u}}{\operatorname{argmin}} \quad & \sum_{k=0}^{N-1} \left(\|\mathbf{x}_k - \mathbf{x}_{k,r}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k - \mathbf{u}_{k,r}\|_{\mathbf{Q}_u}^2 \right) \\ & + \|\mathbf{x}_N - \mathbf{x}_{N,r}\|_{\mathbf{Q}_N} \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad \mathbf{x}_0 = \mathbf{x}_{\text{init}}, \\ & \boldsymbol{\Omega}^B \in [\boldsymbol{\Omega}_{\min}^B, \boldsymbol{\Omega}_{\max}^B], \quad \mathbf{u} \in [\mathbf{u}_{\min}, \mathbf{u}_{\max}], \end{aligned} \quad (10)$$

Note that, we use the thrust of rotors $\mathbf{u}$ defined in (4) and (5) as the control input of NMPC. The state vector is defined as $\mathbf{x} = [\boldsymbol{\xi} \ \dot{\boldsymbol{\xi}} \ \mathbf{q} \ \boldsymbol{\Omega}^B]$, and

$$\mathbf{Q} = \operatorname{diag}(\mathbf{Q}_\xi, \mathbf{Q}_v, \mathbf{Q}_q, \mathbf{Q}_\Omega), \quad \mathbf{Q}_N = \mathbf{Q} \quad (11)$$

The reference state vector $\mathbf{x}_r$ and input $\mathbf{u}_r$ can be obtained from a trajectory planner which generates full states such as the one introduced in [4]. Function $\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$ is the discretized version of nonlinear quadrotor model (1)-(5). The same as many other works (see, e.g., [3, 39]), we omit $\mathbf{G}_2$ and $\mathbf{G}_3$ related terms in (4) as their effects are negligible. $\mathbf{x}_{\text{init}}$ is the current state estimation when solving the OCP. $\mathbf{u}_{\min}$ and $\mathbf{u}_{\max}$ are minimum and maximum values of motor thrusts. Constraints on angular velocities are also added, which is found beneficial for improving the stability of NMPC according to [5]. Note that in the above optimization problem, the following abuse of notation is used when calculating quaternion error:

$$\mathbf{q} - \mathbf{q}_r = (\mathbf{q} \otimes \mathbf{q}_r^{-1})_{x,y,z} \quad (12)$$

$$\mathbf{I}_v \dot{\boldsymbol{\Omega}}^B = \mathbf{I}_v \boldsymbol{\alpha}^B = -\boldsymbol{\Omega}^B \times \mathbf{I}_v \boldsymbol{\Omega}^B + \boldsymbol{\tau} + \mathbf{d}_\tau, \quad (3)$$

其中 $\otimes$ 表示四元数乘法运算符。 $\boldsymbol{\Omega}$ 是 $\mathcal{F}_B$ 相对于 $\mathcal{F}_I$ 的角速度。其导数，即角加速度，表示为 $\boldsymbol{\alpha}$ 。在整篇论文中，我们使用其在 $\mathcal{F}_B$ 上的投影，即 $\boldsymbol{\Omega}^B = [\Omega_x, \Omega_y, \Omega_z]^T$ ，因为它可以直接从惯性测量单元 (IMU) 测量得到。 $\mathbf{I}_v$ 表示整个四旋翼的惯性矩阵。 $\boldsymbol{\tau}$ 是由旋翼产生的总力矩。 $\mathbf{d}_\tau$ 是作用在机体力矩上的模型不确定性，可能来源于高阶空气动力学效应、重心偏差或旋翼之间的差异。

集体推力和旋翼产生的扭矩是旋翼转速的函数：

$$\begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix} = \mathbf{G}_1 \mathbf{u} + \mathbf{G}_2 \dot{\boldsymbol{\omega}} + \mathbf{G}_3(\boldsymbol{\Omega}) \boldsymbol{\omega} \quad (4)$$

哪里

$$\mathbf{u} = c_t \boldsymbol{\omega}^{\circ 2} \quad (5)$$

表示每个旋翼产生的推力， $^\circ$ 表示哈达玛幂。 c_t 是推力系数。 $\boldsymbol{\omega}$ 是每个螺旋桨角速度的向量。 $\mathbf{G}_1$ 到 $\mathbf{G}_3$ 是定义的矩阵

$$\mathbf{G}_1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ l \sin \beta & -l \sin \beta & -l \sin \beta & l \sin \beta \\ -l \cos \beta & -l \cos \beta & l \cos \beta & l \cos \beta \\ c_q/c_t & -c_q/c_t & c_q/c_t & -c_q/c_t \end{bmatrix} \quad (6)$$

$$\mathbf{G}_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ I_p & -I_p & I_p & -I_p \end{bmatrix} \quad (7)$$

$$\mathbf{G}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ I_p \Omega_y & -I_p \Omega_y & I_p \Omega_y & -I_p \Omega_y \\ -I_p \Omega_x & I_p \Omega_x & -I_p \Omega_x & I_p \Omega_x \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (8)$$

其中 c_q 是扭矩系数； β 和 l 是按图 2 定义的几何参数。 I_p 是绕 z_B 的转子惯性。

项 $\mathbf{G}_2 \dot{\boldsymbol{\omega}}$ 和 $\mathbf{G}_3(\boldsymbol{\Omega}) \boldsymbol{\omega}$ 分别是由转子角加速度和陀螺效应引起的转矩，这些通常在控制器设计中被忽略。然而，在第 IV-C 节中，我们将重新审视考虑惯性转矩项 $\mathbf{G}_2 \dot{\boldsymbol{\omega}}$ 影响的角加速度控制 INDI 方法 [13]。

2) *Aerodynamic Drag Model*: 四旋翼在高速飞行过程中会经历显著的空气动力阻力，这需要精确建模以在最小化计算负担的同时提高跟踪精度。在本工作中，我们使用了一种能够捕捉主要效应的空气动力阻力模型，并且在如 [12] 等研究中被证明是有效的。

$$\mathbf{f}_a^B = \begin{bmatrix} -k_{d,x} v_x \\ -k_{d,y} v_y \\ -k_{d,z} v_z + k_h (v_x^2 + v_y^2) \end{bmatrix} \quad (9)$$

其中 $[v_x, v_y, v_z] = \mathbf{R}(\mathbf{q})^T \dot{\boldsymbol{\xi}}$ (即机体坐标系下的速度投影；这里我们假设风速为零)。 $k_{d,x,y,z}$ 和 k_h 是可以从飞行数据识别的正参数。

IV. 方法论

本节详细说明了比较的两种控制方法。选择了一种非线性 NMPC 方法，该方法考虑了每个转子的推力限制、完整的非线性动力学和空气动力学效应。至于 DFBC 方法，我们改进了最新技术，使这些因素也得到考虑，从而确保与 NMPC 的公平比较。最后，这两种方法都通过 INDI 控制器 [15] 得到了增强，以将单个转子的推力指令转换为转子速度指令，同时提高了对旋转动力学模型不确定性和扰动的鲁棒性。两种方法的控制示意图如图 4 和图 3 所示。

A. Nonlinear Model Predictive Controller

NMPC 通过以递推视域的方式求解有限时间最优控制问题 (OCP) 来生成控制指令。给定参考轨迹，代价函数是时间视域内预测状态与参考状态之间的误差，这意味着在时间视域内使用了多个参考点。为了进行数值优化，我们将状态和输入在时间视域 $\tau \in [t, t+h]$ 内离散化为 N 个相等的区间，区间大小为 $dt = h/N$ ，其中 h 表示视域长度，从而得到一个受约束的非线性优化问题：

$$\begin{aligned} \mathbf{u}_{\text{NMPC}} = \underset{\mathbf{u}}{\operatorname{argmin}} \quad & \sum_{k=0}^{N-1} \left(\|\mathbf{x}_k - \mathbf{x}_{k,r}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_k - \mathbf{u}_{k,r}\|_{\mathbf{Q}_u}^2 \right) \\ & + \|\mathbf{x}_N - \mathbf{x}_{N,r}\|_{\mathbf{Q}_N} \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \quad \mathbf{x}_0 = \mathbf{x}_{\text{init}}, \\ & \boldsymbol{\Omega}^B \in [\boldsymbol{\Omega}_{\min}^B, \boldsymbol{\Omega}_{\max}^B], \quad \mathbf{u} \in [\mathbf{u}_{\min}, \mathbf{u}_{\max}], \end{aligned} \quad (10)$$

请注意，我们使用在 (4) 和 (5) 中定义的转子的推力 $\mathbf{u}$ 作为 NMPC 的控制输入。状态向量定义为 $\mathbf{x} = [\boldsymbol{\xi} \ \dot{\boldsymbol{\xi}} \ \mathbf{q} \ \boldsymbol{\Omega}^B]$ ，并且

$$\mathbf{Q} = \operatorname{diag}(\mathbf{Q}_\xi, \mathbf{Q}_v, \mathbf{Q}_q, \mathbf{Q}_\Omega), \quad \mathbf{Q}_N = \mathbf{Q} \quad (11)$$

参考状态向量 $\mathbf{x}_r$ 和输入 $\mathbf{u}_r$ 可以从轨迹规划器中获得，该规划器生成完整状态，例如在 [4] 中介绍的状态。函数 $\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$ 是非线性四旋翼模型 (1)-(5) 的离散版本。与许多其他工作相同 (参见，例如 [3, 39])，我们在 (4) 中省略了与 $\mathbf{G}_2$ 和 $\mathbf{G}_3$ 相关的项，因为它们的影响可以忽略不计。 $\mathbf{x}_{\text{init}}$ 是求解 OCP 时的当前状态估计。 $\mathbf{u}_{\min}$ 和 $\mathbf{u}_{\max}$ 分别是电机推力的最小值和最大值。同时还添加了角速度约束，根据 [5] 的研究，这被发现有助于提高 NMPC 的稳定性。注意，在上述优化问题中，在计算四元数误差时使用了以下符号滥用：

$$\mathbf{q} - \mathbf{q}_r = (\mathbf{q} \otimes \mathbf{q}_r^{-1})_{x,y,z} \quad (12)$$

The above NMPC solves the full nonlinear model of a quadrotor, instead of resorting to a cascaded structure, or linear assumptions. In this paper, this quadratic nonlinear optimization problem is solved by a sequential quadratic programming (SQP) algorithm executed in a real-time iteration scheme [40]. We implement this algorithm using ACADO [41] toolkit with qpOASES [42] as the solver. More implementation details are given in Sec. V.

B. Differential-Flatness Based Controller

Quadrotors are differentially flat systems [25], namely, all the states and inputs can be written as algebraic functions of the flat outputs and their derivatives. This allows a direct mapping from the flat outputs (positions ξ and heading ψ) to the angular rates and angular accelerations, which are leveraged by DFBC as feed-forward terms to improve the tracking accuracy.

In the following, we introduced the DFBC method improved from a previous work [25], where the original geometric attitude controller is replaced by the tilt-prioritized method [31]. We also use the quadratic-programming based control allocation [14] to address input constraints. These modifications are beneficial in tracking dynamically *infeasible* trajectories, as will be discussed in Sec. VI-D. Fig. 4 presents an overview of this method.

1) *Desired Attitude and Collective Thrust*: First of all, we calculate the desired acceleration from a PD controller:

$$\ddot{\xi}_d = K_\xi (\xi_r - \xi) + K_v (\dot{\xi}_r - \dot{\xi}) + \ddot{\xi}_r \quad (13)$$

where K_ξ and K_v are positive-definite diagonal gain matrices. Then from (1) and (9), the desired thrust T_d and thrust direction $z_{B,d}$ are obtained as

$$T_d z_{B,d} = (\ddot{\xi}_d - g) m - R(q) f_a^B \quad (14)$$

Given reference heading angle ψ_r , we get an intermediate axis $x_{C,d}$, by which the desired attitude can be obtained by the following equations:

$$x_{C,d} = [\cos \psi_r, \sin \psi_r, 0]^T, \quad (15)$$

$$y_{B,d} = \frac{z_{B,d} \times x_{C,d}}{\|z_{B,d} \times x_{C,d}\|}, \quad x_{B,d} = y_{B,d} \times z_{B,d} \quad (16)$$

$$R(q_d) = [x_{B,d}, y_{B,d}, z_{B,d}]. \quad (17)$$

where q_d is the desired attitude expressed by the quaternion.

2) *Reference Angular Velocity and Acceleration*: We leverage the differential flatness property of a quadrotor to derive the reference angular velocity and angular accelerations. Using them into the attitude control can help in tracking jerk and snap (3rd and 4th order derivatives of position ξ), which is found beneficial to the tracking performance [13].

Taking the derivative of both sides of (1) and assuming a constant external aerodynamic force f_a , we have

$$m \ddot{\xi} = \dot{T} z_B + T \Omega \times z_B. \quad (18)$$

Then given reference jerk $\ddot{\xi}_r$ and substitute it for $\ddot{\xi}$ in (18), we get

$$h_\Omega \triangleq \Omega \times z_B = (m \ddot{\xi}_r - \dot{T} z_B) / T, \quad (19)$$

by which the reference angular velocity can be obtained as

$$\Omega_r^B = \begin{bmatrix} -h_\Omega \cdot y_B, & h_\Omega \cdot x_B, & \dot{\psi}_r z_I \cdot z_B \end{bmatrix}^T, \quad (20)$$

where ψ_r is the reference heading angle.

We further take the derivative on both sides of (18) and uses snap reference $\ddot{\xi}_r$ to replace $\ddot{\xi}$, yielding

$$\begin{aligned} h_\alpha &\triangleq \dot{\Omega} \times z_B \\ &= \frac{m}{T} \ddot{\xi}_r - \left(\Omega \times (\Omega \times z_B) + \frac{2\dot{T}}{T} \Omega \times z_B + \frac{\ddot{T}}{T} z_B \right). \end{aligned} \quad (21)$$

Then the desired angular acceleration can be obtained as

$$\alpha_r^B = \begin{bmatrix} -h_\alpha \cdot y_B, & h_\alpha \cdot x_B, & \ddot{\psi}_r z_I \cdot z_B \end{bmatrix}^T. \quad (22)$$

Note that in (20) to (22) we use the current attitude $\{x_B, y_B, z_B\}$, angular velocity Ω , and collective thrust T instead of their references. Hence, the drone still follows the reference jerk $\ddot{\xi}_r$ and snap $\ddot{\xi}_r$ even though its attitude, angular rates, and collective thrust have been deviated from the reference.

Above derivations for Ω_r^B and α_r^B need the value of collective thrust T and its derivatives. While T can be calculated from (4)(5) with measured rotor speed ω , its derivatives ($\dot{T}$ and $\ddot{T}$) are unable to be directly measured. For this reason, we approximate them by using reference jerk $\ddot{\xi}_r$ and snap $\ddot{\xi}_r$. Multiplying (dot-product) both sides of (18) by z_B , we have

$$\dot{T} = m \ddot{\xi}_r \cdot z_B \quad (23)$$

and its derivative

$$\ddot{T} = m \ddot{\xi}_r \cdot z_B + m(\Omega \times z_B) \cdot \dot{\xi}_r. \quad (24)$$

Equation (23) and (24) are then substituted into (18)-(22) to calculate Ω_r^B and α_r^B .

3) *Tilt-Prioritized Attitude Control*: Quadrotors use rotor drag torques to achieve heading control. The control effectiveness of heading is around one order of magnitude lower than pitch and roll. As a consequence, heading control is prone to cause motor saturations. Fortunately, the thrust orientation of a quadrotor (namely tilt) is independent of its heading angle. Thus tilt-prioritized control has been proposed in [31] that regulates the reduced-attitude (pitch and roll) error $\tilde{q}_{e,red}$ and yaw error $\tilde{q}_{e,yaw}$ separately as follows:

$$[q_{e,w}, q_{e,x}, q_{e,y}, q_{e,z}]^T = q_d \otimes q^{-1}, \quad (25)$$

$$\tilde{q}_{e,red} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}} \begin{bmatrix} q_{e,w} q_{e,x} - q_{e,y} q_{e,z} \\ q_{e,w} q_{e,y} + q_{e,x} q_{e,z} \\ 0 \end{bmatrix}, \quad (26)$$

$$\tilde{q}_{e,yaw} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}} [0 \quad 0 \quad q_{e,z}]^T. \quad (27)$$

上述 NMPC 求解了四旋翼的完整非线性模型，而不是依赖于级联结构或线性假设。在本文中，这个二次非线性优化问题通过顺序二次规划（SQP）算法在实时迭代方案中求解 [40]。我们使用 ACADO [41] 工具包实现该算法，并使用 qpOASES [42] 作为求解器。更多实现细节见第 V 节。

B. Differential-Flatness Based Controller

四旋翼是微分平坦系统 [25]，即所有状态和输入都可以写成平坦输出及其导数的代数函数。这允许将平坦输出（位置 ξ 和航向 ψ ）直接映射到角速度和角加速度，DFBC 利用这些作为前馈项来提高跟踪精度。

在下文中，我们介绍了从先前工作 [25] 改进的 DFBC 方法，其中原始的几何姿态控制器被倾斜优先方法 [31] 所取代。我们还使用基于二次规划的控制分配 [14] 来处理输入约束。这些修改有助于跟踪动态 *infeasible* 轨迹，如第 VI-D 节所讨论。图 4 展示了该方法的概览。

1) *Desired Attitude and Collective Thrust*: 首先，我们从 PD 控制器计算所需的加速度：

$$\ddot{\xi}_d = K_\xi (\xi_r - \xi) + K_v (\dot{\xi}_r - \dot{\xi}) + \ddot{\xi}_r \quad (13)$$

其中 K_ξ 和 K_v 是正定对角增益矩阵。然后根据 (1) 和 (9)，所需推力 T_d 以及推力方向 $z_{B,d}$ 得到如下

$$T_d z_{B,d} = (\ddot{\xi}_d - g) m - R(q) f_a^B \quad (14)$$

给定参考航向角 ψ_r ，我们得到一个中间轴 $x_{C,d}$ ，通过该轴可以通过以下方程获得所需的姿态：

$$x_{C,d} = [\cos \psi_r, \sin \psi_r, 0]^T, \quad (15)$$

$$y_{B,d} = \frac{z_{B,d} \times x_{C,d}}{\|z_{B,d} \times x_{C,d}\|}, \quad x_{B,d} = y_{B,d} \times z_{B,d} \quad (16)$$

$$R(q_d) = [x_{B,d}, y_{B,d}, z_{B,d}]. \quad (17)$$

其中 q_d 是四元数表达的期望姿态。

2) *Reference Angular Velocity and Acceleration*: 我们利用四旋翼的微分平坦特性来推导参考角速度和角加速度。将它们用于姿态控制有助于跟踪加速度和抖动（位置的三阶和四阶导数 ξ ），这被发现在跟踪性能上是有益的 [13]。

对(1)式的两边求导，并假设外部气动力 f_a 为常量，我们得到

$$m \ddot{\xi} = \dot{T} z_B + T \Omega \times z_B. \quad (18)$$

然后给定参考剪切... ξ_r 并将其代入(18)中的... ξ ，我们得到

$$h_\Omega \triangleq \Omega \times z_B = (m \ddot{\xi}_r - \dot{T} z_B) / T, \quad (19)$$

通过该方法可以获得参考角速度为

$$\Omega_r^B = \begin{bmatrix} -h_\Omega \cdot y_B, & h_\Omega \cdot x_B, & \dot{\psi}_r z_I \cdot z_B \end{bmatrix}^T, \quad (20)$$

其中 ψ_r 是参考航向角。

我们进一步对 (18) 的两边求导，并使用快照参考... ξ_r 替换... ξ ，得到

$$\begin{aligned} h_\alpha &\triangleq \dot{\Omega} \times z_B \\ &= \frac{m}{T} \ddot{\xi}_r - \left(\Omega \times (\Omega \times z_B) + \frac{2\dot{T}}{T} \Omega \times z_B + \frac{\ddot{T}}{T} z_B \right). \end{aligned} \quad (21)$$

那么可以得到所需的角加速度为

$$\alpha_r^B = \begin{bmatrix} -h_\alpha \cdot y_B, & h_\alpha \cdot x_B, & \ddot{\psi}_r z_I \cdot z_B \end{bmatrix}^T. \quad (22)$$

请注意，在(20)到(22)中，我们使用的是当前的姿态

$\{x_B, y_B, z_B\}$ 、角速度 Ω 和总推力 T ，而不是它们的参考值。因此，即使无人机的姿态、角速度和总推力偏离了参考值，它仍然会跟随参考加速度... ξ_r 和加速度... ξ_r 。

上述关于 Ω_r^B 和 α_r^B 的推导需要集体推力 T 及其导数的数值。虽然 T 可以通过测量的转子速度 ω 从 (4)(5) 计算得到，但其导数 ($\dot{T}$ 和 $\ddot{T}$) 无法直接测量。因此，我们通过使用参考加速度... ξ_r 和加速度... ξ_r 对其进行近似。将 (18) 的两边与 z_B 相乘（点积），我们得到

$$\dot{T} = m \ddot{\xi}_r \cdot z_B \quad (23)$$

及其导数

$$\ddot{T} = m \ddot{\xi}_r \cdot z_B + m(\Omega \times z_B) \cdot \ddot{\xi}_r. \quad (24)$$

然后将方程 (23) 和 (24) 代入 (18) - (22) 以计算 Ω_r^B 和 α_r^B 。

3) *Tilt-Prioritized Attitude Control*: 四旋翼利用旋翼阻力矩来实现航向控制。航向的控制有效性大约比俯仰和横滚低一个数量级。因此，航向控制容易导致电机饱和。幸运的是，四旋翼的推力方向（即倾斜）与其航向角是独立的。因此，[31] 提出了倾斜优先控制，该方法分别调节简化姿态（俯仰和横滚）误差 $\tilde{q}_{e,red}$ 和偏航误差 $\tilde{q}_{e,yaw}$ ，具体如下：

$$[q_{e,w}, q_{e,x}, q_{e,y}, q_{e,z}]^T = q_d \otimes q^{-1}, \quad (25)$$

$$\tilde{q}_{e,red} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}} \begin{bmatrix} q_{e,w} q_{e,x} - q_{e,y} q_{e,z} \\ q_{e,w} q_{e,y} + q_{e,x} q_{e,z} \\ 0 \end{bmatrix}, \quad (26)$$

$$\tilde{q}_{e,yaw} = \frac{1}{\sqrt{q_{e,w}^2 + q_{e,z}^2}} [0 \quad 0 \quad q_{e,z}]^T. \quad (27)$$

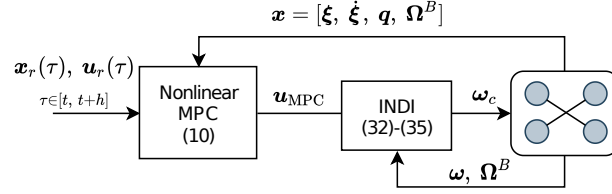


Fig. 3: The control diagram of the model predictive controller with an INDI inner-loop controller.

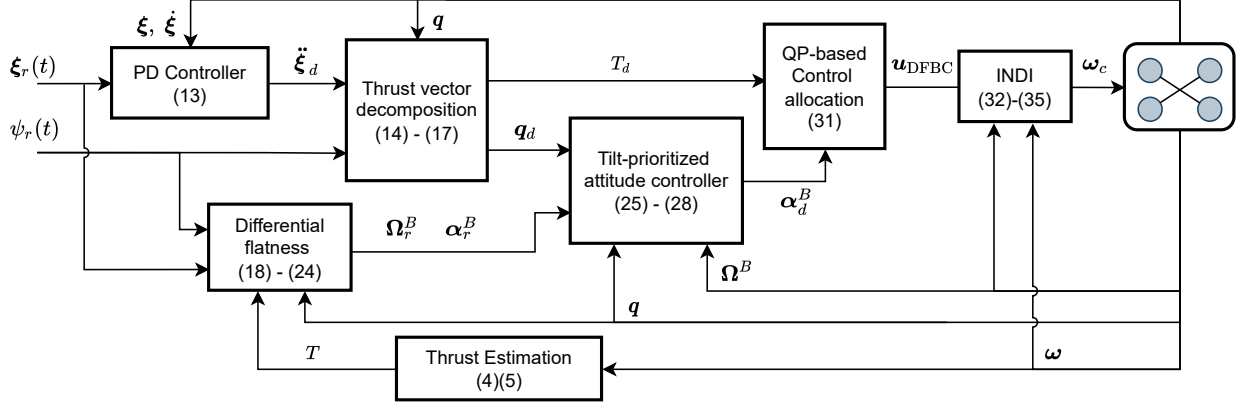


Fig. 4: The control diagram of the differential-flatness-based controller, combined with an INDI inner-loop controller.

Subsequently, the desired angular acceleration can be obtained by the following attitude control law:

$$\alpha_d^B = k_{q,red} \tilde{q}_{e,red} + k_{q,yaw} \text{sgn}(q_{e,w}) \tilde{q}_{e,yaw} + K_\Omega (\Omega_r^B - \Omega^B) + \alpha_r^B \quad (28)$$

where $k_{q,red}$ and $k_{e,yaw}$ are positive gains for reduced-attitude and yaw control respectively. Setting a relatively high $k_{q,red}$ over $k_{e,yaw}$ is helpful in improving position tracking accuracy while preventing input saturations. α_r^B in (28) performs as a feed-forward term. It is worth noting that, the inclusion of α_r^B seems to be reasonable from a theoretical perspective [31], although removing it has been found to have almost no effect in our real-world experiments.

4) *Quadratic-Programming-Based Control Allocation*: The control allocation module generates thrust commands of each individual rotor from desired collective thrust T_d and angular acceleration α_d^B . The same as NMPC, we also neglect the G_2 and G_3 terms in (4). Then from (3) and (4), we obtain the direct-inversion control allocation:

$$u = G_1^{-1} \begin{bmatrix} T_d \\ I_v \alpha_d^B + \Omega^B \times I_v \Omega^B \end{bmatrix}, \quad (29)$$

$$u_{DFBC} = \max(u_{\min}, \min(u, u_{\max})) \quad (30)$$

This, however, does not consider input limits and may cause loss of control. For instance, an excessive collective thrust command can saturate all motors and consequently disable the attitude control.

An effective alternative to address saturations is the quadratic-programming based allocation that solves the fol-

lowing optimization problem:

$$u_{DFBC} = \underset{u}{\operatorname{argmin}} \left\| \begin{bmatrix} T_d \\ I_v \alpha_d^B + \Omega^B \times I_v \Omega^B \end{bmatrix} - G_1 u \right\|_W \quad (31)$$

s.t. $u \in [u_{\min}, u_{\max}]$,

where $W \in \mathbb{R}^{4 \times 4}$ is a positive-definite diagonal weight matrix. Each diagonal entry respectively indicates the weight on the thrust, pitch, roll and yaw channels. Generally, setting a relatively high weight on pitch and roll (see Table. I) is advantageous to prevent quadrotor loss-of-control when motor saturations are inevitable (e.g., tracking dynamically infeasible trajectories). If the solution is originally within control bounds, the result is the same as the direct-inversion allocation from (30). As for the implementation details, we solve this quadratic programming problem using an Active-Set Method from the qpOASES solver [42].

C. Incremental Nonlinear Dynamic Inversion

After obtaining thrust commands from (10) or (31), one can use (5) to directly obtain the rotor speed command. However, the above-mentioned controllers neglects the unmodeled term d_τ in the rotational dynamics (3), which are found detrimental to the overall control performance.

Modeling d_τ is very challenging for real-world systems. Therefore, we resort to incremental nonlinear dynamic inversion (INDI), a sensor-based controller that uses instantaneous sensor measurement, instead of an explicit model, to represent system dynamics, thus being robust against model uncertainties and external disturbances. The performance and robustness of INDI have been demonstrated in previous research (see, e.g., [15, 13, 16]) with proven stability [43].

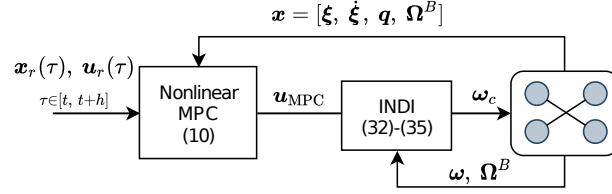


图3: 具有INDI内环控制器的模型预测控制器的控制示意图。

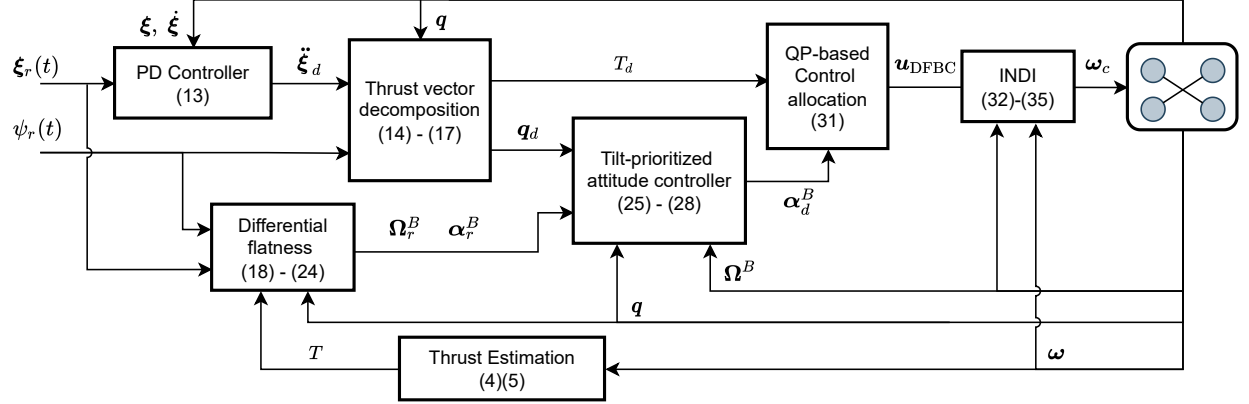


图4: 基于微分平坦性的控制器控制图, 结合了INDI内环控制器。

随后, 可以通过以下姿态控制律获得所需的角加速度:

$$\alpha_d^B = k_{q,red} \tilde{q}_{e,red} + k_{q,yaw} \text{sgn}(q_{e,w}) \tilde{q}_{e,yaw} + K_\Omega (\Omega_r^B - \Omega^B) + \alpha_r^B \quad (28)$$

其中 $k_{q,red}$ 和 $k_{e,yaw}$ 分别是用于减小姿态和偏航控制的正增益。将相对较高的 $k_{q,red}$ 设置于 $k_{e,yaw}$ 之上, 有助于提高位置跟踪精度, 同时防止输入饱和。(28) 中的 α_r^B 作为前馈项运行。值得注意的是, 从理论角度来看, 引入 α_r^B 似乎是合理的 [31], 尽管在我们的实际实验中发现移除它几乎没有影响。

4) *Quadratic-Programming-Based Control Allocation*: 控制分配模块根据期望的总推力 T_d 和角加速度 α_d^B 生成每个独立转子的推力指令。与 NMPC 相同, 我们也忽略了 (4) 中的 G_2 和 G_3 项。然后, 从 (3) 和 (4) 中, 我们得到直接反演的控制分配:

$$u = G_1^{-1} \begin{bmatrix} T_d \\ I_v \alpha_d^B + \Omega^B \times I_v \Omega^B \end{bmatrix}, \quad (29)$$

$$u_{DFBC} = \max(u_{\min}, \min(u, u_{\max})) \quad (30)$$

然而, 这并未考虑输入限制, 并可能导致失控。例如, 过大的总推力指令可能使所有电机饱和, 从而使姿态控制失效。

解决饱和问题的有效替代方法是基于二次规划的分配, 它可以求解以下问题——

以下优化问题:

$$u_{DFBC} = \arg \min_u \left\| \begin{bmatrix} T_d \\ I_v \alpha_d^B + \Omega^B \times I_v \Omega^B \end{bmatrix} - G_1 u \right\|_W \quad (31)$$

s.t. $u \in [u_{\min}, u_{\max}]$,

其中 $W \in \mathbb{R}^{4 \times 4}$ 是一个正定对角权重矩阵。每个对角线元素分别表示对推力、俯仰、横滚和偏航通道的权重。通常, 为俯仰和横滚设置相对较高的权重 (见表 I) 有利于在电机饱和和不可避免时 (例如, 跟踪动态不可行轨迹) 防止四旋翼失控。如果解最初在控制范围内, 结果与公式 (30) 的直接反演分配相同。关于实现细节, 我们使用 qpOA SES 求解器 [42] 中的主动集方法求解这个二次规划问题。

C. Incremental Nonlinear Dynamic Inversion

在从 (10) 或 (31) 获得推力指令后, 可以使用 (5) 直接获得旋翼转速指令。然而, 上述控制器忽略了旋转动力学 (3) 中未建模的项 d_τ , 而这被发现对整体控制性能有不利影响。

对实际系统而言, 建模 d_τ 非常具有挑战性。因此, 我们采用增量非线性动态反演 (INDI), 这是一种基于传感器的控制器, 它使用瞬时传感器测量, 而不是显式模型, 来表示系统动力学, 从而对模型不确定性和外部干扰具有鲁棒性。INDI 的性能和鲁棒性已在以往研究中得到验证 (参见例如 [15, 13, 16]), 并且稳定性已被证明 [43]。

We use INDI as the inner-loop controller of both NMPC and DFBC for fair comparisons. The hybridization of INDI and DFBC is similar to a related work [13], except for the attitude controller and control allocation introduced in the previous section. Here, a method is proposed to effectively combine INDI with NMPC to improve its robustness against rotational model uncertainties.

After knowing the single rotor thrust command $\mathbf{u}_{\text{DFBC}}$ or $\mathbf{u}_{\text{NMPC}}$ from (31) and (10) respectively, we can retrieve the desired collective thrust, and desired angular acceleration using (3) and (4), yielding

$$\begin{bmatrix} \hat{T}_d \\ \mathbf{I}_v \hat{\boldsymbol{\alpha}}_d^B + \boldsymbol{\Omega}^B \times \mathbf{I}_v \boldsymbol{\Omega}^B \end{bmatrix} = \mathbf{G}_1 \mathbf{u}_{\text{DFBC/NMPC}}. \quad (32)$$

Note that, for the DFBC method, $\hat{T}_d$ and $\hat{\boldsymbol{\alpha}}_d^B$ are different from those derived from (14) and (28) if the optimal cost of (31) is non-zero. Then from INDI, we can get the desired body torque (see (30) and (31) of [13] for detailed derivations)

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_f + \mathbf{I}_v \left(\hat{\boldsymbol{\alpha}}_d^B - \dot{\boldsymbol{\Omega}}_f^B \right) \quad (33)$$

Hence, the effect of unmodeled $\mathbf{d}_\tau$ is captured by filtered angular acceleration measurement $\dot{\boldsymbol{\Omega}}_f^B$ and filtered body torque $\boldsymbol{\tau}_f$, where

$$\boldsymbol{\tau}_f = \bar{\mathbf{G}}_1 c_t \boldsymbol{\omega}_f^{\circ 2} + \Delta t^{-1} \bar{\mathbf{G}}_2 c_t (\boldsymbol{\omega}_f - \boldsymbol{\omega}_{f,k-1}) \quad (34)$$

is calculated from rotor speed measurements. $\boldsymbol{\Omega}_f^B$ and $\boldsymbol{\omega}_f$ are low-pass filtered body-rate and rotor speeds with the same cut-off frequencies. Thus they have a similar amount of delay and can be synchronized. In this paper, we use a second-order Butterworth filter with a 12 Hz cut-off frequency. Note that we use subscript $k-1$ to indicate the last sampled variable, and Δt is the sampling interval. $\bar{\mathbf{G}}_1$ and $\bar{\mathbf{G}}_2$ respectively indicate matrices formed by the last three rows of $\mathbf{G}_1$ and $\mathbf{G}_2$.

Then from (14), the following equation is obtained to solve rotor speed command $\boldsymbol{\omega}_c$:

$$\begin{bmatrix} \hat{T}_d \\ \boldsymbol{\tau}_d \end{bmatrix} = \mathbf{G}_1 c_t \boldsymbol{\omega}_c^{\circ 2} + \Delta t^{-1} \mathbf{G}_2 c_t (\boldsymbol{\omega}_c - \boldsymbol{\omega}_{c,k-1}), \quad (35)$$

with the only unknown $\boldsymbol{\omega}_c$ which can be solved numerically. Differently from [13], the motor time constant is not needed in (35).

The INDI inner-loop controller converts the original thrust inputs from the high-level controller to rotor speed commands. Because this conversion is through algebraic equations, system delay typically seen in the cascaded control structures can be effectively circumvented. The advantage of INDI over classical PID inner-loop controller will be demonstrated in Sec. VII.

V. IMPLEMENTATION DETAILS

Before elaborating on the simulation and real-world experiments, we introduce the implementation details and experimental setups. Both NMPC and DFBC flight controllers are implemented in an open-sourced flight stack *Agilicious*¹ programmed in C++, with ACADO [41] toolkit and

qpOASES [42] as external libraries. A wrapper in ROS environment is also written which allows data logging, interfacing, and network communication. The controller gains are listed in Table I and the inertial and geometric parameters of the tested quadrotor are listed in Table II. These parameters are used for both simulation and real-world experiments.

In the simulation experiments, the flight software runs on a laptop at 300 Hz while the frequency of NMPC is limited to 100 Hz for consistency with the real-world experiments. The rotor speed command is sent to a simulator included in *Agilicious* that uses a 4th-order Runge-Kutta integrator to propagate quadrotor dynamic equations (1)-(4) at 500 Hz. To simulate the motor dynamics, rotor speed commands generated by the controllers pass through a low-pass filter with a 30 ms time constant. The drag model (9) is also included to study the effect of aerodynamics, of which the parameters are included in Table II. The quadrotor states from the simulator are directly fed into the controllers unless a dedicated state estimation error is simulated.

The real-world experiment is performed in an indoor arena equipped with a motion capture system (VICON), with a $30 \times 30 \times 8 \text{ m}^3$ tracking volume, as is shown in Fig. 1. The flight software runs at 300 Hz on an onboard NVIDIA Jetson TX2 embedded computer. It includes the control algorithms (DFBC/NMPC and INDI inner-loop), and an extended Kalman filter fusing VICON (400 Hz) and IMU measurements (500 Hz) to obtain the full-state estimates. While DFBC runs at 300 Hz, the frequency of NMPC is limited to 100 Hz due to its computational complexity. The INDI inner-loop also runs at 300 Hz no matter which controller is being used. The onboard computer sends rotor speed commands of individual rotors to a low-level RadixFC board. The latter runs a customized firmware based on the open-source NuttX real-time operating system at 500 Hz to perform closed-loop rotor speed control, and sends rotor speed and IMU measurements to the onboard computer. We refer the users to *Agilicious* for more details about the hardware.

TABLE I: Controller gains and parameters

NMPC		DFBC	
$\mathbf{Q}_\xi$	diag(200, 200, 500)	$\mathbf{K}_\xi$	diag(10, 10, 10)
$\mathbf{Q}_v$	diag(1, 1, 1)	$\mathbf{K}_v$	diag(6, 6, 6)
$\mathbf{Q}_q$	diag(5, 5, 200)	$(k_{q,red}, k_{q,yaw})$	(150, 3)
$\mathbf{Q}_\Omega$	diag(1, 1, 1)	$\mathbf{K}_\Omega$	diag(20, 20, 8)
$\mathbf{Q}_u$	diag(6, 6, 6, 6)	$\mathbf{W}$	diag(0.001, 10, 10, 0.1)
dt	50 ms		
N	20		

TABLE II: Quadrotor Configurations

Parameter(s)		Value(s)
m	[kg]	0.75
l	[m]	0.14
β	[deg]	56
$\mathbf{I}_v$	[gm ²]	diag(2.5, 2.1, 4.3)
$(u_{\min}, u_{\max})$	[N]	(0, 8.5)
c_q	[Nm ²]	$2.37e^{-8}$
c_t	[N ²]	$1.51e^{-6}$
$(k_{d,x}, k_{d,y}, k_{d,z})$	[kg/s]	(0.26, 0.28, 0.42)
k_h	[kg/m]	0.01

¹<https://agilicious.dev/>

我们将 INDI 用作 NMPC 和 DFBC 的内环控制器，以便进行公平的比较。INDI 与 DFBC 的混合类似于相关研究 [13]，只是有所不同的是前一节介绍的姿态控制器和控制分配方法。在这里，提出了一种方法，将 INDI 有效地与 NMPC 结合，以提高其对旋转模型不确定性的鲁棒性。

在分别从 (31) 和 (10) 得到单旋翼推力指令 $\mathbf{u}_{\text{DFBC}}$ 或 $\mathbf{u}_{\text{NMPC}}$ 后，我们可以使用 (3) 和 (4) 来获取期望的总推力和期望角加速度，得到

$$\begin{bmatrix} \hat{T}_d \\ \mathbf{I}_v \hat{\alpha}_d^B + \boldsymbol{\Omega}^B \times \mathbf{I}_v \boldsymbol{\Omega}^B \end{bmatrix} = \mathbf{G}_1 \mathbf{u}_{\text{DFBC/NMPC}}. \quad (32)$$

请注意，对于 DFBC 方法，如果 (31) 的最优代价非零，那么 $\hat{T}_d$ 和 $\hat{\alpha}_d^B$ 与由 (14) 和 (28) 推导出的值不同。然后通过 INDI，我们可以得到期望的机体扭矩（详细推导见 [13] 的 (30) 和 (31)）

$$\boldsymbol{\tau}_d = \boldsymbol{\tau}_f + \mathbf{I}_v (\hat{\alpha}_d^B - \dot{\boldsymbol{\Omega}}_f^B) \quad (33)$$

因此，未建模的 $\mathbf{d}_\tau$ 的影响通过滤波后的角加速度测量 $\dot{\boldsymbol{\Omega}}_f^B$ 和滤波后的机体力矩 $\boldsymbol{\tau}_f$ 来捕捉，其中

$$\boldsymbol{\tau}_f = \bar{\mathbf{G}}_1 c_t \boldsymbol{\omega}_f^{\circ 2} + \Delta t^{-1} \bar{\mathbf{G}}_2 c_t (\boldsymbol{\omega}_f - \boldsymbol{\omega}_{f,k-1}) \quad (34)$$

是根据转子速度测量计算得出的。 $\boldsymbol{\Omega}_f^B$ 和 $\boldsymbol{\omega}_f$ 是经过低通滤波的机身角速度和转子速度，具有相同的截止频率。因此它们具有相似的延迟时间，可以同步。在本文中，我们使用截止频率为 12 Hz 的二阶巴特沃斯滤波器。注意，我们使用下标 $k-1$ 表示最后采样的变量， Δt 是采样间隔。 $\bar{\mathbf{G}}_1$ 和 $\bar{\mathbf{G}}_2$ 分别表示由 $\mathbf{G}_1$ 和 $\mathbf{G}_2$ 的最后三行形成的矩阵。

然后由 (14) 式得到以下方程来求解转子速度指令 $\boldsymbol{\omega}_c$ ：

$$\begin{bmatrix} \hat{T}_d \\ \boldsymbol{\tau}_d \end{bmatrix} = \mathbf{G}_1 c_t \boldsymbol{\omega}_c^{\circ 2} + \Delta t^{-1} \mathbf{G}_2 c_t (\boldsymbol{\omega}_c - \boldsymbol{\omega}_{c,k-1}), \quad (35)$$

唯一未知的 $\boldsymbol{\omega}_c$ 可以通过数值方法求解。与 [13] 不同，(35) 中不需要电机时间常数。

INDI 内环控制器将高层控制器的原始推力输入转换为转子速度指令。由于这种转换是通过代数方程完成的，因此可以有效规避通常在级联控制结构中看到的系统延迟。INDI 相对于经典 PID 内环控制器的优势将在第七节中展示。

五、实施细节

在详细说明仿真和实际实验之前，我们介绍了实现细节和实验设置。NMPC 和 DFBC 飞行控制器都在一个开源飞行堆栈 *Agilicious*¹ 中实现，该堆栈使用 C++ 编程，并配合 ACADO [41] 工具包。

¹<https://agilicious.dev/>

qpOASES [42] 作为外部库。在 ROS 环境中也编写了一个封装器，允许数据记录、接口和网络通信。控制器增益列在表 I 中，测试四旋翼的惯性和几何参数列在表 II 中。这些参数用于仿真和实际实验。

在模拟实验中，飞行软件以 300 Hz 的频率在笔记本电脑上运行，而 NMPC 的频率被限制为 100 Hz，以与实际实验保持一致。旋翼速度指令发送到包含在 *Agilicious* 中的模拟器，该模拟器使用 4 阶 Runge-Kutta 积分器以 500 Hz 的频率传播四旋翼动力学方程 (1)-(4)。为了模拟电机动力学，由控制器生成的旋翼速度指令通过一个时间常数为 30 毫秒的低通滤波器。还包括阻力模型 (9) 来研究气动效应，其参数列在表 II 中。模拟器中的四旋翼状态直接输入控制器，除非模拟了专门的状态估计误差。

现实世界的实验在一个配备运动捕捉系统 (VICON) 的室内场地中进行，跟踪体积为 $30 \times 30 \times 8 \text{ m}^3$ ，如图 1 所示。飞行软件在车载 NVIDIA Jetson TX2 嵌入式计算机上以 300 Hz 运行。它包括控制算法 (DFBC/NMPC 和 INDI 内环) 以及扩展卡尔曼滤波器，用于融合 VICON (400 Hz) 和 IMU 测量 (500 Hz) 以获得完整状态估计。虽然 DFBC 以 300 Hz 运行，NMPC 的频率由于计算复杂性被限制在 100 Hz。无论使用哪种控制器，INDI 内环也以 300 Hz 运行。车载计算机向低级 RadixFC 板发送各个转子速度指令。后者以 500 Hz 运行基于开源 NuttX 实时操作系统的定制固件，以执行闭环转子速度控制，并将转子速度和 IMU 测量发送回车载计算机。我们建议用户参考 *Agilicious* 以获取更多详细信息。

表 I: 控制器增益和参数

NMPC		DFBC	
$\mathbf{Q}_\xi$	diag(200, 200, 500)	$\mathbf{K}_\xi$	diag(10, 10, 10)
$\mathbf{Q}_v$	diag(1, 1, 1)	$\mathbf{K}_v$	diag(6, 6, 6)
$\mathbf{Q}_q$	diag(5, 5, 200)	$(k_{q,red}, k_{q,yaw})$	(150, 3)
$\mathbf{Q}_\Omega$	diag(1, 1, 1)	$\mathbf{K}_\Omega$	diag(20, 20, 8)
$\mathbf{Q}_u$	diag(6, 6, 6, 6)	$\mathbf{W}$	diag(0.001, 10, 10, 0.1)
dt	50 ms		
N	20		

表二: 四旋翼配置

Parameter(s)		Value(s)
m	[kg]	0.75
l	[m]	0.14
β	[deg]	56
$\mathbf{I}_v$	[gm ²]	diag(2.5, 2.1, 4.3)
$(u_{\min}, u_{\max})$	[N]	(0, 8.5)
c_q	[Nm ²]	$2.37e^{-8}$
c_t	[N ²]	$1.51e^{-6}$
$(k_{d,x}, k_{d,y}, k_{d,z})$	[kg/s]	(0.26, 0.28, 0.42)
k_h	[kg/m]	0.01

VI. SIMULATION EXPERIMENTS

In a set of controlled experiments in simulation, we examine DFBC and NMPC to address the following research questions: (i) Given the computational budget and a well-identified model, which control approach achieves better tracking performance? (ii) How do these two approaches perform in tracking dynamically *infeasible* trajectories, which inevitably lead to actuator saturation? (iii) How do these two approaches perform in simulated practical situations with a model mismatch, external disturbances, state estimation latency? Note that in this section, both methods are tested with the augmentation of the INDI inner-loop controller. In this section, we use NMPC and DFBC to represent those with INDI and an aerodynamic drag model for readability.

A. Reference Trajectories

Multiple elliptical reference trajectories on both horizontal and vertical planes are generated for testing the tracking performance. The results of tracking these simple trajectory primitives can further reflect the performance of tracking more complex 3D trajectories. These trajectories are parameterized by the maximum velocity $V_{\max}$, maximum acceleration $a_{\max}$, and ellipticity n . Specifically, the horizontal reference trajectories are expressed as:

$$\xi_r(t) = [r_{\max} \sin(kt), r_{\min} \cos(kt), 5]^T; \quad (36)$$

and the vertical reference trajectories are

$$\xi_r(t) = [r_{\max} \sin(kt), 0, 5 + r_{\min} \cos(kt)]^T \quad (37)$$

where

$$r_{\max} = V_{\max}^2 / a_{\max}, \quad k = a_{\max} / V_{\max}, \quad r_{\min} = r_{\max} / n. \quad (38)$$

The heading angle of each horizontal reference trajectory is selected to always point at the center of the ellipse; for vertical trajectories, the reference heading angles are kept constant. Specifically, 144 reference trajectories are generated by combining parameters $a_{\max} \in \{10, 20, 30, 40, 50, 60\} \text{ m/s}^2$, $V_{\max} \in \{5, 10, 15, 20\} \text{ m/s}$, $n \in \{1, 2, 5\}$.

Tracking some of these trajectories may require the quadrotor to exceed its thrust limitations. These trajectories are referred to as dynamically *infeasible*. In many time-critical applications, such as autonomous drone racing, rotors need to reach their full thrust limits to fully exploit the agility of the platform and achieve faster lap times. Designing such an agile time-optimal trajectory requires a perfect model that captures real thrust limits and aerodynamic effects in high-speed flights. Without this model, the generated trajectory may exceed the quadrotor capabilities and becomes dynamically *infeasible* to accurately follow. Since such an accurate model is usually unattainable, studying the performance when tracking these *infeasible* trajectories is necessary.

For the simulated quadrotor with the configurations given in Table II, there are 68 *infeasible* trajectories and 76 *feasible* trajectories. They are determined by being tracked by a modified NMPC without thrust limits imposed. If the applied thrust exceeds the maximum thrust of the real drone, the trajectory is marked as *infeasible*. The feasibility of each reference trajectory is given in Table III

TABLE III: Dynamical feasibility of reference trajectories. (*feasible*:✓; *infeasible*:✗)

(a) Horizontal trajectories.												
$a_{\max}$ [m/s ²]	$n = 1$				$n = 2$				$n = 5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓
30	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✓	✓
40	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗	✓
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

(b) Vertical trajectories.												
$a_{\max}$ [m/s ²]	$n = 1$				$n = 2$				$n = 5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
30	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
40	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

B. Evaluation Criteria

In the following comparisons, we use the root mean square error (RMSE) of position and heading as the precision metric of a method. The crash rate is another criterion to show the robustness of a method. A certain flight is defined as *crashed* if its position ξ violates the following spatial constraint at an arbitrary instant:

$$\inf\{\xi_r(t)\} - \mathbf{b} \leq \xi \leq \sup\{\xi_r(t)\} + \mathbf{b}. \quad (39)$$

We select $\mathbf{b} = [5, 5, 5]^T$ meters in this study.

C. Tracking Dynamically Feasible Trajectories

First of all, we compare the performance of NMPC and DFBC in tracking 76 *feasible* trajectories. We perform the test in an ideal condition with perfect model knowledge and state estimates. Since only *feasible* trajectories are tracked, there is no saturation of single rotor thrusts and both methods succeed in tracking all trajectories without a single crash.

In this set of tests, we fine-tune the parameters of both methods (listed in Table. I) such that they achieve a similar position tracking error. Fig. 5 compares the boxplots of NMPC and DFBC when tracking trajectories with different reference maximum accelerations. Both methods have similar position RMSE in these flights. As for the heading tracking, NMPC has an average heading RMSE of 2.0 deg, which is better than 5.8 deg of DFBC.

D. Tracking Dynamically Infeasible Trajectories

1) *Tracking Accuracy*: Fig. 6 shows the box plot of position and heading RMSE of NMPC and DFBC in tracking the previously generated 68 *infeasible* trajectories. Crashed flights are excluded from this plot (see Fig. 7 for the crash rate). We find that the DFBC method shows smaller position RMSE in tracking trajectories with lower acceleration. However, as the reference acceleration grows, NMPC significantly outperforms

VI. 模拟实验

在一系列受控的模拟实验中，我们研究 DFBC 和 NMPC 以解决以下研究问题：(i) 在计算预算充足且模型良好识别的情况下，哪种控制方法能够实现更好的跟踪性能？(ii) 这两种方法在跟踪动态 *infeasible* 轨迹时的表现如何，而这种轨迹不可避免地会导致执行器饱和？(iii) 在模拟的实际情况下，包括模型不匹配、外部扰动和状态估计延迟，这两种方法的表现如何？注意，在本节中，两种方法均在 INDI 内环控制器的增强下进行测试。本节中，为了可读性，我们用 NMPC 和 DFBC 来表示那些具有 INDI 和空气动力学阻力模型的方法。

A. Reference Trajectories

在水平面和垂直面上生成多个椭圆形参考轨迹，以测试跟踪性能。跟踪这些简单轨迹基元的结果可以进一步反映跟踪更复杂三维轨迹的性能。这些轨迹通过最大速度 $V_{\max}$ 、最大加速度 $a_{\max}$ 和椭圆率 n 进行参数化。具体来说，水平参考轨迹表示为：

$$\xi_r(t) = [r_{\max} \sin(kt), r_{\min} \cos(kt), 5]^T; \quad (36)$$

而垂直参考轨迹是

$$\xi_r(t) = [r_{\max} \sin(kt), 0, 5 + r_{\min} \cos(kt)]^T \quad (37)$$

哪里

$$r_{\max} = V_{\max}^2 / a_{\max}, \quad k = a_{\max} / V_{\max}, \quad r_{\min} = r_{\max} / n. \quad (38)$$

每条水平参考轨迹的航向角选择为始终指向椭圆的中心；对于垂直轨迹，参考航向角保持不变。具体来说，通过组合参数 $a_{\max} \in \{10, 20, 30, 40, 50, 60\} \text{ m/s}^2$ ， $V_{\max} \in \{5, 10, 15, 20\} \text{ m/s}$ ， $n \in \{1, 2, 5\}$ ，生成了 144 条参考轨迹。

跟踪其中一些轨迹可能需要四旋翼超出其推力限制。这些轨迹被称为动态 *infeasible*。在许多时间关键的应用中，例如自主无人机竞速，旋翼需要达到其最大推力极限，以充分利用平台的敏捷性并实现更快的圈速。设计如此敏捷的时间最优轨迹需要一个完美的模型，该模型能够捕捉高速飞行中的实际推力限制和气动效应。没有这个模型，生成的轨迹可能会超出四旋翼的能力，并变得动态 *infeasible* 而难以精确跟踪。由于这样的精确模型通常难以获得，因此研究跟踪这些 *infeasible* 轨迹时的性能是必要的。

对于表 II 中给出的配置的模拟四旋翼，有 68 条 *infeasible* 轨迹和 76 条 *feasible* 轨迹。它们是通过被没有施加推力限制的修改 NMPC 追踪来确定的。如果所施加的推力超过了实际无人机的最大推力，该轨迹将被标记为 *infeasible*。每条参考轨迹的可行性见表 III。

表 III：参考轨迹的动态可行性。(feasible: ✓; infeasible: ✗) (a) 水平轨迹。

$a_{\max}$ [m/s ²]	$n = 1$				$n = 2$				$n = 5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓
30	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✓	✓
40	✓	✓	✓	✓	✗	✓	✓	✓	✗	✗	✗	✓
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

(b) Vertical trajectories.

$a_{\max}$ [m/s ²]	$n = 1$				$n = 2$				$n = 5$			
	$V_{\max}$ [m/s]				$V_{\max}$ [m/s]				$V_{\max}$ [m/s]			
	5	10	15	20	5	10	15	20	5	10	15	20
10	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
20	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
30	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
40	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
50	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
60	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

B. Evaluation Criteria

在以下比较中，我们使用位置和航向的均方根误差 (RMSE) 作为方法的精度指标。撞机率是显示方法鲁棒性的另一个标准。如果某次飞行在任意时刻其位置 ξ 违反以下空间约束，则该飞行被定义为 *crashed*：

$$\inf\{\xi_r(t)\} - \mathbf{b} \leq \xi \leq \sup\{\xi_r(t)\} + \mathbf{b}. \quad (39)$$

在本研究中，我们选择了 $\mathbf{b} = [5, 5, 5]^T$ 米。

C. Tracking Dynamically Feasible Trajectories

首先，我们比较了 NMPC 和 DFBC 在跟踪 76 条 *feasible* 轨迹时的性能。我们在理想条件下进行测试，假设模型知识和状态估计完全准确。由于仅跟踪 *feasible* 条轨迹，因此单个旋翼推力没有饱和，且两种方法都成功跟踪了所有轨迹，没有一次坠机。

在这一组测试中，我们对两种方法的参数（列在表 I 中）进行了微调，以使它们达到相似的位置跟踪误差。图 5 比较了 NMPC 和 DFBC 在跟踪不同参考最大加速度轨迹时的箱线图。在这些飞行中，两种方法的位置均方根误差相似。至于航向跟踪，NMPC 的平均航向均方根误差为 2.0 度，优于 DFBC 的 5.8 度。

D. Tracking Dynamically Infeasible Trajectories

1) *Tracking Accuracy*: 图 6 显示了 NMPC 和 DFBC 在跟踪之前生成的 68 条 *infeasible* 轨迹时的位置和航向 RMSE 箱线图。坠毁的飞行已从此图中排除（坠毁率见图 7）。我们发现，DFBC 方法在跟踪加速度较低的轨迹时显示出较小的位置 RMSE。然而，随着参考加速度的增加，NMPC 表现显著优于 DFBC。

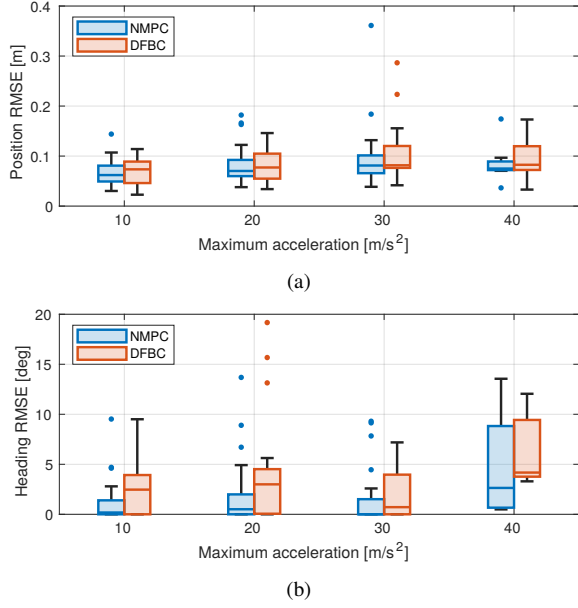


Fig. 5: Boxplot of tracking error (RMSE) in tracking different dynamically *feasible* trajectories (76 in total) categorized by maximum accelerations.

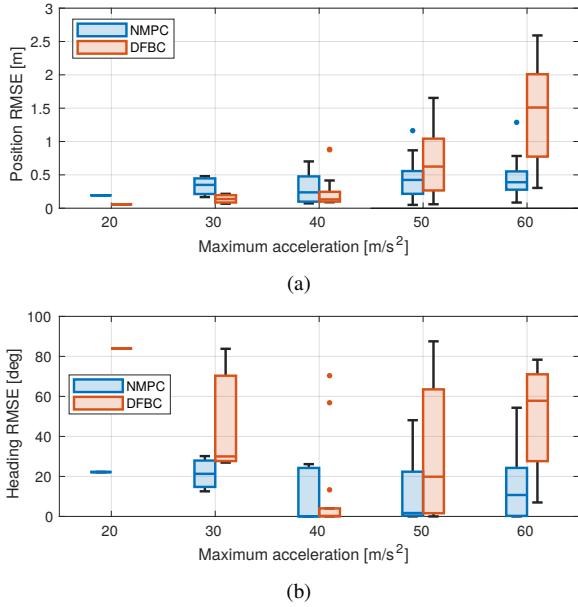


Fig. 6: Boxplot of tracking error (RMSE) in tracking different dynamically *infeasible* trajectories (68 in total) categorized by maximum accelerations. Crashed flights are removed from these analyses (see Fig. 7 for the crash rate).

DFBC. In general, NMPC outperforms DFBC by 48% on position tracking (0.40 m vs. 0.77 m). As for the heading tracking, NMPC has a noticeably less (62%) RMSE than the DFBC method (12.7 deg vs. 33.4 deg). The advantage of NMPC over DFBC becomes particularly relevant if the reference acceleration exceeds the maximum thrust ($\sim 5g$) of the tested quadrotor.

2) *Crash Rates*: Fig. 7 compares the crash rates of DFBC and NMPC in tracking all the *infeasible* trajectories. NMPC (solid-blue) outperforms DFBC (red-dash-dot), especially in

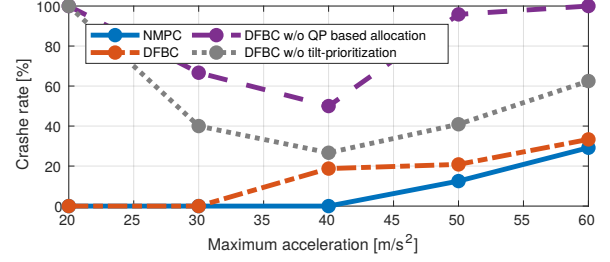


Fig. 7: Crash rates of NMPC, DFBC, and DFBC with different setups: DFBC without quadratic-programming based allocation, DFBC without tilt-prioritized attitude controller but using the geometric attitude controller.

tracking trajectories with high reference accelerations.

Here, we also demonstrate contributions of the tilt-prioritized attitude controller and the quadratic-programming based allocation presented in Sec. IV. We replace them respectively with the conventional *geometric attitude controller* [24] and the *direct-inversion allocation* (30). The resultant crash rates are shown in Fig. 7 as well. It is obvious that the DFBC method tested in this study shows a significantly lower crash rate in tracking *infeasible* trajectories.

E. Robustness Study

While previous simulations are carried out in an ideal condition, this section studies the robustness of NMPC and DFBC methods in the following non-ideal conditions:

- Model mismatch, including the center of gravity bias, mass mismatch, thrust and torque coefficients model mismatch, and error of aerodynamic drag model.
- External disturbances. In details, we simulate constant external forces along x_I , or external torques along both x_B and y_B . These disturbances last for 5 seconds.
- Position, velocity, and attitude estimation latency. This aims at studying the effect of latency from such as signal transmission, state estimation algorithms, and sensors. As angular rates are measured directly by IMU with negligible latency, we only study the latency on pose and velocity estimates.

These robustness studies are conducted using only the *feasible* trajectories since NMPC already shows its advantage over DFBC in tracking *infeasible* trajectories. We set the position RMSE as 5.0 m and heading RMSE as 90 deg for those crashed flights.

Table IV and Table V respectively present the results without and with INDI inner-loop. NMPC shows substantially higher robustness than DFBC against model uncertainties and disturbances on rotational dynamics due to CoG bias or external moments. Adding an INDI inner-loop controller can improve the robustness against these uncertainties. Even so, NMPC still slightly outperforms DFBC when both are augmented by an INDI inner-loop controller.

Unlike the rotational dynamics, NMPC shows a higher crash rate while experiencing model uncertainties and disturbances acting on the translational dynamics. For example, when the real mass is 30% higher, both controllers generate fewer thrusts

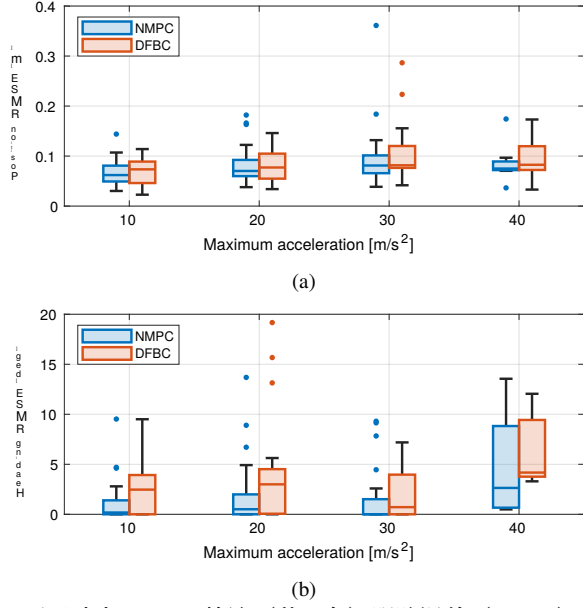


图5: 不同动态 *feasible* 轨迹 (共76条) 跟踪误差 (RMSE) 箱线图, 按照最大加速度分类。

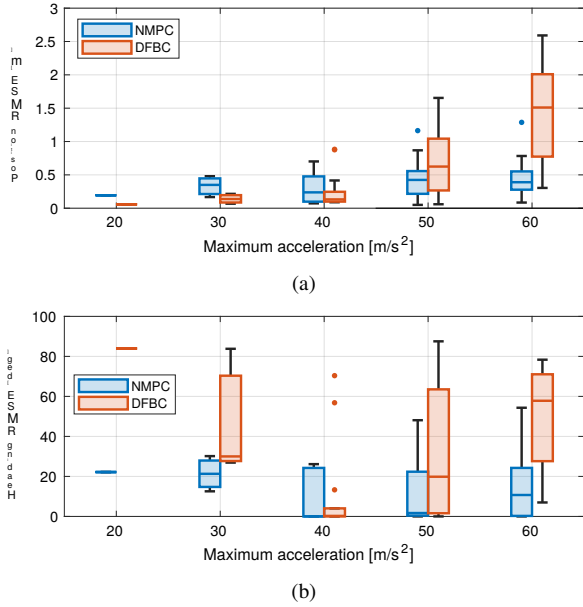


图6: 不同动态 *infeasible* 轨迹 (共68条) 跟踪误差 (RMSE) 的箱线图, 按最大加速度分类。坠毁的飞行已从这些分析中移除 (坠毁率见图7)。

DFBC。总体而言, NMPC 在位置跟踪上的表现比 DFBC 高出 48% (0.40 米对 0.77 米)。在航向跟踪方面, NMPC 的 RMSE 显著低于 DFBC 方法 62% (12.7 度对 33.4 度)。当参考加速度超过测试四旋翼的最大推力 ($\sim 5g$) 时, NMPC 相对于 DFBC 的优势尤其明显。

2) *Crash Rates*: 图7比较了DFBC和NMPC在跟踪所有 *infeasible* 轨迹中的碰撞率。NMPC (实线-蓝色) 优于DFBC (红色-点划线), 特别是在

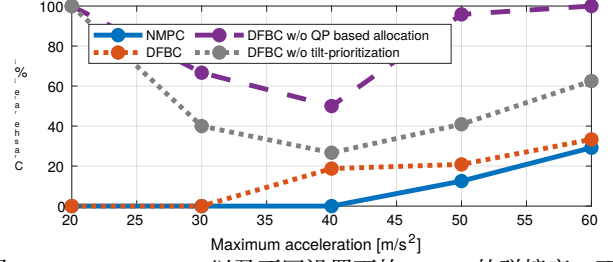


图7: NMPC、DFBC 以及不同设置下的 DFBC 的碰撞率: 不使用基于二次规划分配的 DFBC、不使用倾斜优先姿态控制器但使用几何姿态控制器的 DFBC。

跟踪具有高参考加速度的轨迹。

在这里, 我们还展示了在第四节中提出的倾斜优先姿态控制器和基于二次规划的分配的贡献。我们分别将它们替换为传统的 *geometric attitude controller* [24] 和 *direct-inversion allocation* (30)。结果的碰撞率也如图 7 所示。显然, 本研究测试的 DFBC 方法在跟踪 *infeasible* 轨迹时显示出显著更低的碰撞率。

E. Robustness Study

虽然之前的仿真是在理想条件下进行的, 但本节研究了 NMPC 和 DFBC 方法在以下非理想条件下的鲁棒性:

- 模型不匹配, 包括重心偏差、质量不匹配、推力和力矩系数模型不匹配, 以及空气动力阻力模型的误差。
- 外部干扰。具体来说, 我们模拟沿 x_I 的恒定外力, 或沿 x_B 和 y_B 的外部力矩。这些干扰持续5秒钟。
- 位置、速度和姿态估计的延迟。此处旨在研究来自信号传输、状态估计算法和传感器等的延迟影响。由于角速度由 IMU 直接测量且延迟可忽略不计, 我们仅研究姿态和速度估计的延迟。

这些稳健性研究仅使用 *feasible* 轨迹进行, 因为 NMPC 在跟踪 *infeasible* 轨迹方面已经显示出其相对于 DFBC 的优势。对于那些坠毁的飞行, 我们将位置均方根误差设为 5.0 米, 航向均方根误差设为 90 度。

表 IV 和表 V 分别显示了没有和有 INDI 内环的结果。由于重心偏差或外部力矩引起的旋转动力学模型不确定性和扰动, NMPC 的鲁棒性明显高于 DFBC。添加 INDI 内环控制器可以提高对这些不确定性的鲁棒性。即便如此, 当两者都增加 INDI 内环控制器时, NMPC 仍略优于 DFBC。

与旋转动力学不同, NMPC在经历作用于平移动力学的模型不确定性和干扰时显示出更高的碰撞率。例如, 当实际质量高出30%时, 两种控制器产生的推力都更少

TABLE IV: Tracking performance comparisons in different types of non-ideal conditions commonly seen in practice **without INDI inner loop**. Each number shows the mean and standard deviation (SD) across tracking 76 dynamically *feasible* trajectories.

	Position RMSE [m] (mean $\pm$ SD)		Heading RMSE [deg] (mean $\pm$ SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	0.084 $\pm$ 0.049	0.084 $\pm$ 0.041	2.13 $\pm$ 3.38	5.91 $\pm$ 11.90	0	0
+50% Drag	0.183 $\pm$ 0.066	0.194 $\pm$ 0.084	2.11 $\pm$ 3.35	8.56 $\pm$ 15.46	0	0
+100% Drag	0.286 $\pm$ 0.109	0.305 $\pm$ 0.139	2.25 $\pm$ 3.38	9.07 $\pm$ 11.44	0	0
-30% Mass	0.320 $\pm$ 0.105	0.477 $\pm$ 0.109	2.27 $\pm$ 3.38	11.26 $\pm$ 17.77	0	0
+30% Mass	0.809 $\pm$ 1.455	0.687 $\pm$ 1.001	11.77 $\pm$ 27.21	7.21 $\pm$ 14.11	10.7	0
10% CoG bias	0.264 $\pm$ 0.094	1.499 $\pm$ 1.802	2.67 $\pm$ 3.42	25.95 $\pm$ 35.69	0	20
15% CoG bias	0.464 $\pm$ 0.557	3.039 $\pm$ 2.216	4.58 $\pm$ 10.94	53.62 $\pm$ 41.95	1.3	56
-30% c_t	0.939 $\pm$ 1.415	0.998 $\pm$ 1.216	12.30 $\pm$ 27.13	8.79 $\pm$ 18.01	10.7	0
+30% c_t	0.245 $\pm$ 0.085	0.36 $\pm$ 0.076	2.05 $\pm$ 3.34	6.63 $\pm$ 10.87	0	0
-30% c_q	0.085 $\pm$ 0.052	0.084 $\pm$ 0.041	2.03 $\pm$ 3.37	6.23 $\pm$ 13.03	0	0
+30% c_q	0.084 $\pm$ 0.051	0.084 $\pm$ 0.04	2.07 $\pm$ 3.38	5.81 $\pm$ 11.81	0	0
5N Ext. force	0.252 $\pm$ 0.042	0.302 $\pm$ 0.031	5.08 $\pm$ 3.58	17.42 $\pm$ 14.59	0	0
10N Ext. force	0.716 $\pm$ 1.019	0.689 $\pm$ 0.524	12.59 $\pm$ 19.07	24.59 $\pm$ 16.28	5.3	1.3
15N Ext. force	1.870 $\pm$ 1.892	1.391 $\pm$ 1.069	32.13 $\pm$ 35.38	32.54 $\pm$ 20.58	26.7	6.7
0.1Nm Ext. moment	0.153 $\pm$ 0.038	0.270 $\pm$ 0.098	3.26 $\pm$ 3.35	12.04 $\pm$ 13.45	0	0
0.2Nm Ext. moment	0.319 $\pm$ 0.545	2.538 $\pm$ 2.079	5.85 $\pm$ 10.71	54.89 $\pm$ 31.08	1.3	41.3
0.3Nm Ext. moment	0.566 $\pm$ 0.907	4.949 $\pm$ 0.439	10.87 $\pm$ 17.27	89.16 $\pm$ 6.88	4.0	98.7
10ms Latency	0.085 $\pm$ 0.065	0.097 $\pm$ 0.099	2.07 $\pm$ 3.34	5.79 $\pm$ 11.05	0	0
30ms Latency	0.158 $\pm$ 0.183	0.333 $\pm$ 0.976	5.04 $\pm$ 10.47	13.73 $\pm$ 25.87	0	4.0
50ms Latency	3.450 $\pm$ 2.217	0.669 $\pm$ 1.247	65.00 $\pm$ 38.65	54.82 $\pm$ 30.02	66.7	5.3

TABLE V: Tracking performance comparisons in different types of non-ideal conditions commonly seen in practice **with INDI inner loop**. Each number shows the mean and standard deviation (SD) across tracking 76 dynamically *feasible* trajectories.

	Position RMSE [m] (mean $\pm$ SD)		Heading RMSE [deg] (mean $\pm$ SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	0.082 $\pm$ 0.047	0.085 $\pm$ 0.043	2.03 $\pm$ 3.35	5.83 $\pm$ 11.40	0	0
+50% Drag	0.184 $\pm$ 0.067	0.194 $\pm$ 0.084	2.10 $\pm$ 3.33	8.48 $\pm$ 14.50	0	0
+100% Drag	0.288 $\pm$ 0.11	0.306 $\pm$ 0.139	2.25 $\pm$ 3.32	9.32 $\pm$ 11.99	0	0
-30% Mass	0.320 $\pm$ 0.106	0.477 $\pm$ 0.108	2.26 $\pm$ 3.36	10.42 $\pm$ 14.19	0	0
+30% Mass	0.747 $\pm$ 1.372	0.686 $\pm$ 0.999	10.60 $\pm$ 25.66	7.11 $\pm$ 13.97	9.3	0
10% CoG bias	0.081 $\pm$ 0.047	0.099 $\pm$ 0.104	2.09 $\pm$ 3.32	6.46 $\pm$ 13.65	0	0
15% CoG bias	0.083 $\pm$ 0.051	0.378 $\pm$ 1.117	2.05 $\pm$ 3.30	10.70 $\pm$ 22.52	0	5.3
-30% c_t	1.362 $\pm$ 1.764	1.004 $\pm$ 1.212	19.98 $\pm$ 33.88	10.06 $\pm$ 19.06	18.7	1.3
+30% c_t	0.244 $\pm$ 0.085	0.359 $\pm$ 0.076	2.13 $\pm$ 3.52	6.14 $\pm$ 9.58	0	0
-30% c_q	0.082 $\pm$ 0.049	0.086 $\pm$ 0.047	2.08 $\pm$ 3.35	6.01 $\pm$ 11.69	0	0
+30% c_q	0.082 $\pm$ 0.046	0.085 $\pm$ 0.046	2.07 $\pm$ 3.34	5.83 $\pm$ 11.61	0	0
5N Ext. force	0.254 $\pm$ 0.051	0.302 $\pm$ 0.033	5.03 $\pm$ 3.48	17.58 $\pm$ 14.77	0	0
10N Ext. force	0.725 $\pm$ 1.018	0.690 $\pm$ 0.524	12.38 $\pm$ 19.02	24.81 $\pm$ 16.28	5.3	1.3
15N Ext. force	1.872 $\pm$ 1.891	1.393 $\pm$ 1.070	32.14 $\pm$ 35.35	32.97 $\pm$ 20.80	26.7	6.7
0.1Nm Ext. moment	0.083 $\pm$ 0.048	0.087 $\pm$ 0.049	2.49 $\pm$ 3.22	7.95 $\pm$ 12.46	0	0
0.2Nm Ext. moment	0.087 $\pm$ 0.051	0.113 $\pm$ 0.088	3.19 $\pm$ 3.35	13.65 $\pm$ 17.25	0	0
0.3Nm Ext. moment	0.298 $\pm$ 0.962	0.764 $\pm$ 1.663	8.40 $\pm$ 17.29	26.93 $\pm$ 29.41	4.0	13.3
10ms Latency	0.087 $\pm$ 0.088	0.099 $\pm$ 0.106	2.21 $\pm$ 3.44	5.73 $\pm$ 11.65	0	0
30ms Latency	0.280 $\pm$ 0.802	0.265 $\pm$ 0.800	8.39 $\pm$ 20.12	14.34 $\pm$ 25.85	2.7	2.7
50ms Latency	3.480 $\pm$ 2.235	0.694 $\pm$ 1.294	64.02 $\pm$ 39.96	52.29 $\pm$ 29.12	68.0	6.7

than required to track the trajectory, resulting in a constant position tracking error. With this tracking error, NMPC fails to converge in over 10% of flights and crashes the drone. The same reason also explains the significantly higher crash rate of NMPC in the presence of large external disturbances. Fig. 8 shows an example where NMPC failed in converging and crashed the drone after experiencing a 10 N external force disturbance. Note that adding INDI inner-loop does not improve the performance of either controller to reject disturbances in the translational dynamics. A previous research [13] implemented INDI in the position control (INDI outer-loop)

for DFBC by virtue of its cascaded structure and improved its translational robustness. On the contrary, hybridizing INDI outer-loop with NMPC seems challenging owing to its non-cascaded nature.

The two methods also perform differently in the presence of system latency. While NMPC slightly outperforms DFBC when the system latency is lower than 30 ms, as the latency grows, NMPC shows a much higher crash rate (68.0%) than DFBC (6.7%). As Fig. 9 shows, we repeat the tests under 30 ms and 50 ms latency with a high-gain setup and a low-gain setup. In both setups, these gains are tuned such that

制表第四章：在实际常见的不同类型非理想条件下，无内置INDI的跟踪性能比较循环。每个数字显示了追踪76条动态feasible轨迹的平均值和标准偏差（SD）。

	Position RMSE [m] (mean $\pm$ SD)		Heading RMSE [deg] (mean $\pm$ SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	0.084 $\pm$ 0.049	0.084 $\pm$ 0.041	2.13 $\pm$ 3.38	5.91 $\pm$ 11.90	0	0
+50% Drag	0.183 $\pm$ 0.066	0.194 $\pm$ 0.084	2.11 $\pm$ 3.35	8.56 $\pm$ 15.46	0	0
+100% Drag	0.286 $\pm$ 0.109	0.305 $\pm$ 0.139	2.25 $\pm$ 3.38	9.07 $\pm$ 11.44	0	0
-30% Mass	0.320 $\pm$ 0.105	0.477 $\pm$ 0.109	2.27 $\pm$ 3.38	11.26 $\pm$ 17.77	0	0
+30% Mass	0.809 $\pm$ 1.455	0.687 $\pm$ 1.001	11.77 $\pm$ 27.21	7.21 $\pm$ 14.11	10.7	0
10% CoG bias	0.264 $\pm$ 0.094	1.499 $\pm$ 1.802	2.67 $\pm$ 3.42	25.95 $\pm$ 35.69	0	20
15% CoG bias	0.464 $\pm$ 0.557	3.039 $\pm$ 2.216	4.58 $\pm$ 10.94	53.62 $\pm$ 41.95	1.3	56
-30% c_t	0.939 $\pm$ 1.415	0.998 $\pm$ 1.216	12.30 $\pm$ 27.13	8.79 $\pm$ 18.01	10.7	0
+30% c_t	0.245 $\pm$ 0.085	0.36 $\pm$ 0.076	2.05 $\pm$ 3.34	6.63 $\pm$ 10.87	0	0
-30% c_q	0.085 $\pm$ 0.052	0.084 $\pm$ 0.041	2.03 $\pm$ 3.37	6.23 $\pm$ 13.03	0	0
+30% c_q	0.084 $\pm$ 0.051	0.084 $\pm$ 0.04	2.07 $\pm$ 3.38	5.81 $\pm$ 11.81	0	0
5N Ext. force	0.252 $\pm$ 0.042	0.302 $\pm$ 0.031	5.08 $\pm$ 3.58	17.42 $\pm$ 14.59	0	0
10N Ext. force	0.716 $\pm$ 1.019	0.689 $\pm$ 0.524	12.59 $\pm$ 19.07	24.59 $\pm$ 16.28	5.3	1.3
15N Ext. force	1.870 $\pm$ 1.892	1.391 $\pm$ 1.069	32.13 $\pm$ 35.38	32.54 $\pm$ 20.58	26.7	6.7
0.1Nm Ext. moment	0.153 $\pm$ 0.038	0.270 $\pm$ 0.098	3.26 $\pm$ 3.35	12.04 $\pm$ 13.45	0	0
0.2Nm Ext. moment	0.319 $\pm$ 0.545	2.538 $\pm$ 2.079	5.85 $\pm$ 10.71	54.89 $\pm$ 31.08	1.3	41.3
0.3Nm Ext. moment	0.566 $\pm$ 0.907	4.949 $\pm$ 0.439	10.87 $\pm$ 17.27	89.16 $\pm$ 6.88	4.0	98.7
10ms Latency	0.085 $\pm$ 0.065	0.097 $\pm$ 0.099	2.07 $\pm$ 3.34	5.79 $\pm$ 11.05	0	0
30ms Latency	0.158 $\pm$ 0.183	0.333 $\pm$ 0.976	5.04 $\pm$ 10.47	13.73 $\pm$ 25.87	0	4.0
50ms Latency	3.450 $\pm$ 2.217	0.669 $\pm$ 1.247	65.00 $\pm$ 38.65	54.82 $\pm$ 30.02	66.7	5.3

制表 V：在实际中常见的不同类型非理想条件下，使用 INDI 内环进行的跟踪性能比较。每个数字显示了在追踪76条动态feasible轨迹过程中计算的平均值和标准偏差（SD）。

	Position RMSE [m] (mean $\pm$ SD)		Heading RMSE [deg] (mean $\pm$ SD)		Crash Rate [%]	
	NMPC	DFBC	NMPC	DFBC	NMPC	DFBC
Baseline	0.082 $\pm$ 0.047	0.085 $\pm$ 0.043	2.03 $\pm$ 3.35	5.83 $\pm$ 11.40	0	0
+50% Drag	0.184 $\pm$ 0.067	0.194 $\pm$ 0.084	2.10 $\pm$ 3.33	8.48 $\pm$ 14.50	0	0
+100% Drag	0.288 $\pm$ 0.11	0.306 $\pm$ 0.139	2.25 $\pm$ 3.32	9.32 $\pm$ 11.99	0	0
-30% Mass	0.320 $\pm$ 0.106	0.477 $\pm$ 0.108	2.26 $\pm$ 3.36	10.42 $\pm$ 14.19	0	0
+30% Mass	0.747 $\pm$ 1.372	0.686 $\pm$ 0.999	10.60 $\pm$ 25.66	7.11 $\pm$ 13.97	9.3	0
10% CoG bias	0.081 $\pm$ 0.047	0.099 $\pm$ 0.104	2.09 $\pm$ 3.32	6.46 $\pm$ 13.65	0	0
15% CoG bias	0.083 $\pm$ 0.051	0.378 $\pm$ 1.117	2.05 $\pm$ 3.30	10.70 $\pm$ 22.52	0	5.3
-30% c_t	1.362 $\pm$ 1.764	1.004 $\pm$ 1.212	19.98 $\pm$ 33.88	10.06 $\pm$ 19.06	18.7	1.3
+30% c_t	0.244 $\pm$ 0.085	0.359 $\pm$ 0.076	2.13 $\pm$ 3.52	6.14 $\pm$ 9.58	0	0
-30% c_q	0.082 $\pm$ 0.049	0.086 $\pm$ 0.047	2.08 $\pm$ 3.35	6.01 $\pm$ 11.69	0	0
+30% c_q	0.082 $\pm$ 0.046	0.085 $\pm$ 0.046	2.07 $\pm$ 3.34	5.83 $\pm$ 11.61	0	0
5N Ext. force	0.254 $\pm$ 0.051	0.302 $\pm$ 0.033	5.03 $\pm$ 3.48	17.58 $\pm$ 14.77	0	0
10N Ext. force	0.725 $\pm$ 1.018	0.690 $\pm$ 0.524	12.38 $\pm$ 19.02	24.81 $\pm$ 16.28	5.3	1.3
15N Ext. force	1.872 $\pm$ 1.891	1.393 $\pm$ 1.070	32.14 $\pm$ 35.35	32.97 $\pm$ 20.80	26.7	6.7
0.1Nm Ext. moment	0.083 $\pm$ 0.048	0.087 $\pm$ 0.049	2.49 $\pm$ 3.22	7.95 $\pm$ 12.46	0	0
0.2Nm Ext. moment	0.087 $\pm$ 0.051	0.113 $\pm$ 0.088	3.19 $\pm$ 3.35	13.65 $\pm$ 17.25	0	0
0.3Nm Ext. moment	0.298 $\pm$ 0.962	0.764 $\pm$ 1.663	8.40 $\pm$ 17.29	26.93 $\pm$ 29.41	4.0	13.3
10ms Latency	0.087 $\pm$ 0.088	0.099 $\pm$ 0.106	2.21 $\pm$ 3.44	5.73 $\pm$ 11.65	0	0
30ms Latency	0.280 $\pm$ 0.802	0.265 $\pm$ 0.800	8.39 $\pm$ 20.12	14.34 $\pm$ 25.85	2.7	2.7
50ms Latency	3.480 $\pm$ 2.235	0.694 $\pm$ 1.294	64.02 $\pm$ 39.96	52.29 $\pm$ 29.12	68.0	6.7

跟踪轨迹所需的能力不足，导致恒定的位置跟踪误差。由于这种跟踪误差，NMPC在超过10%的飞行中无法收敛并导致无人机坠毁。同样的原因也解释了在存在较大外部扰动时NMPC显著更高的坠机率。图8展示了一个例子，其中NMPC未能收敛，并在经历10 N外力扰动后坠毁无人机。注意，增加INDI内环并不能改善任一控制器在平移动力学中抗扰动的性能。先前的研究[13]在位置控制中实现了INDI（INDI外环）

由于其级联结构，DFBC 提高了其平移鲁棒性。相反，由于 INDI 外环的非级联特性，将其与 NMPC 混合似乎具有挑战性。

这两种方法在存在系统延迟的情况下表现也不同。当系统延迟低于30毫秒时，NMPC略优于DFBC，但随着延迟增加，NMPC的碰撞率（68.0%）远高于DFBC（6.7%）。如图9所示，我们在30毫秒和50毫秒延迟下重复测试，分别使用高增益设置和低增益设置。在这两种设置中，这些增益都经过调节，以便

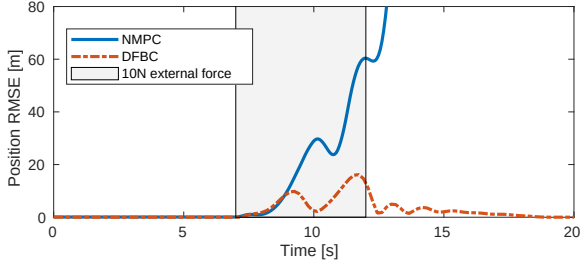


Fig. 8: Position tracking error for a loop trajectory ($V_{max} = 15m/s$, $a_{max} = 40m/s^2$, $n = 1$). Grey area indicates the period with 10 N lateral disturbances acting on the drone. NMPC failed to converge and crashed the drone, while the DFBC method succeeds in recovering the drone after the external disturbance disappeared.

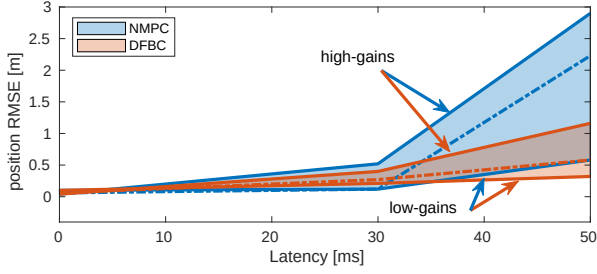


Fig. 9: Position RMSE of NMPC and DFBC in the presence of different amounts of estimation latency, with different sets of control gains. Results using gains of previous studies are shown in dash-dot lines. NMPC shows higher sensitivity to the gain selections when estimation latency appears.

both controllers have identical average position RMSE in the ideal condition. As is expected, reducing gains will alleviate the effect of estimation latency. Interestingly, we observe that NMPC is more sensitive to the gain selections when system latency is larger than 30 ms.

F. Effect of Controller Parameters

Controller performance is dominated by the parameters or gains. However, finding optimal controller gains can be tedious and highly dependent on the drone parameters. Thus this section performs a sensitivity analysis to provide insights into the effect of controller parameters on the tracking performance. We set the control parameters given in Table I as the baseline, and compare the tracking performance with altered parameters with respect to the baseline.

1) *Nonlinear MPC*: Fig. 10 shows the effect of changing elements of the weight matrix Q . The position tracking performance is highly correlated with Q_ξ and Q_v . Among all the parameters, the position weight Q_ξ plays the most important role. A larger penalty on the position error results in a smaller tracking error. However, as Q_ξ continues growing, the NMPC starts to destabilize the drone. We believe that this is due to the presence of actuator dynamics in the simulation, which introduces system delay and causes instability with high gain controllers. Note that in real-world experiments, system latency can be larger due to state estimation algorithms and signal transmissions, which further reduces the permitted maximum Q_ξ . Another observation is that increasing Q_Ω

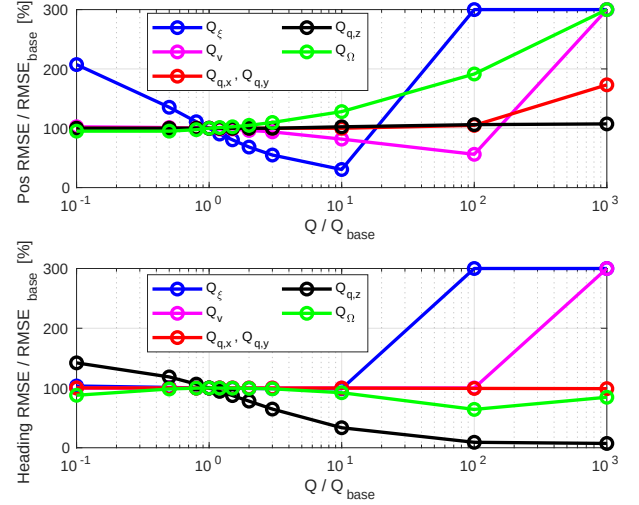


Fig. 10: Position (top) and heading (bottom) tracking RMSE of the NMPC method in different control parameters. The vertical axis represents the ratio between RMSE and that from a baseline parameter setup. The horizontal axis represents the ratio between the control parameter and the baseline parameter. The RMSE of those crashed flights are clamped at 300% of $RMSE_{base}$.

also adversarially affects the position tracking performance as it dilutes the power of position weights. Regarding heading control, as is expected, increasing $Q_{q,z}$ can improve the heading tracking performance while causing little effect on position tracking.

2) *Differential Flatness Based Control*: The controller gains of DFBC consists of position, attitude gains and weight matrix for control allocation. The gains are selected to make the position error $\xi_e = \xi - \xi_r$ a second-order system:

$$\ddot{\xi}_e + 2\zeta_\xi \omega_{n,\xi} \dot{\xi}_e + \omega_{n,\xi}^2 \xi_e = 0 \quad (40)$$

where ζ_ξ and $\omega_{n,\xi}$ are damping ratio and natural frequency, respectively. Then substituting (13) into (40) and replacing ξ_d with $\tilde{\xi}$ (assuming a perfect inner-loop attitude control), we can calculate the position control gains as follows:

$$K_\xi = \text{diag}(\omega_{n,\xi}^2, \omega_{n,\xi}^2, \omega_{n,\xi}^2), \quad K_v = 2\zeta_\xi \sqrt{K_\xi}. \quad (41)$$

Similarly, the attitude gains can be designed as:

$$k_{q,x} = k_{q,y} = \omega_{n,xy}^2, \quad k_{q,z} = \omega_{n,z}^2, \quad (42)$$

$$K_\Omega = 2\zeta_q \text{diag}(\sqrt{k_{q,x}}, \sqrt{k_{q,y}}, \sqrt{k_{q,z}}). \quad (43)$$

where ζ_q is the damping ratio, $\omega_{n,xy}$ and $\omega_{n,z}$ are desired natural frequencies of reduced attitude and yaw errors respectively.

Fig. 11 presents the relationship between tracking error and closed-loop natural frequency. Both time constants and tracking errors are normalized by the baseline gains given in Table I. A higher frequency indicates a higher control gain, leading to less tracking error. However, high gains are also prone to suffer from system delay and subsequently cause instability.

Weighting matrix W is selected based on tuning in the simulation without sophisticated calculation. It aims at leaving

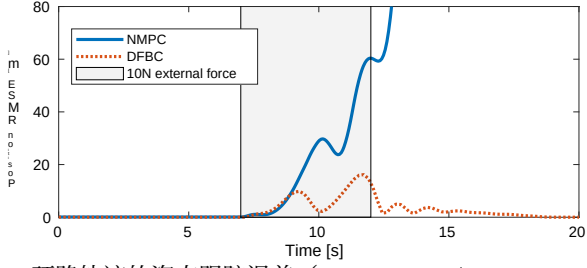


图8: 环路轨迹的姿态跟踪误差 ($V_{max} = 15m/s$, $a_{max} = 40$ 米/秒², $n = 1$)。灰色区域表示无人机受到10 N横向扰动的时间段。NMPC未能收敛并导致无人机坠毁, 而DFBC方法在外部扰动消失后成功使无人机恢复。

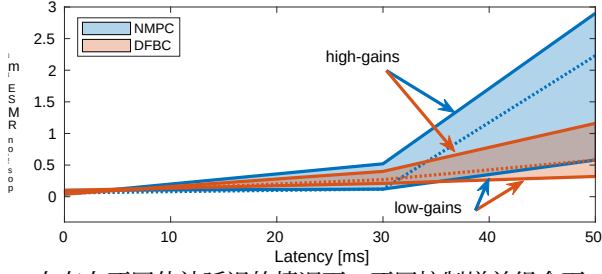


图9: 在存在不同估计延迟的情况下, 不同控制增益组合下NMP C和DFBC的位置均方根误差 (RMSE)。使用前人研究增益的结果用点划线表示。当存在估计延迟时, NMPC对增益选择表现出更高的敏感性。

在理想条件下, 两种控制器的平均位置均方根误差 (RMS E) 相同。如预期所示, 降低增益将缓解估算延迟的影响。有趣的是, 我们观察到当系统延迟大于30毫秒时, NMP C对增益选择更为敏感。

F. Effect of Controller Parameters

控制器的性能主要取决于参数或增益。然而, 寻找最佳控制器增益可能非常繁琐, 并且高度依赖于无人机的参数。因此, 本节进行灵敏度分析, 以提供控制器参数对跟踪性能影响的洞察。我们将表 I 中给出的控制参数设为基线, 并将修改参数后的跟踪性能与基线进行比较。

1) *Nonlinear MPC*: 图10显示了更改权重矩阵元素 Q 的效果。位置跟踪性能与 Q_ξ 和 Q_v 高度相关。在所有参数中, 位置权重 Q_ξ 起着最重要的作用。对位置误差的惩罚越大, 跟踪误差越小。然而, 随着 Q_ξ 的持续增加, NMPC开始使无人机不稳定。我们认为这是由于仿真中存在执行器动态, 它引入了系统延迟, 并导致高增益控制器的不稳定。需要注意的是, 在实际实验中, 由于状态估计算法和信号传输, 系统延迟可能更大, 这进一步降低了允许的最大 Q_ξ 。另一个观察结果是增加 Q_Ω

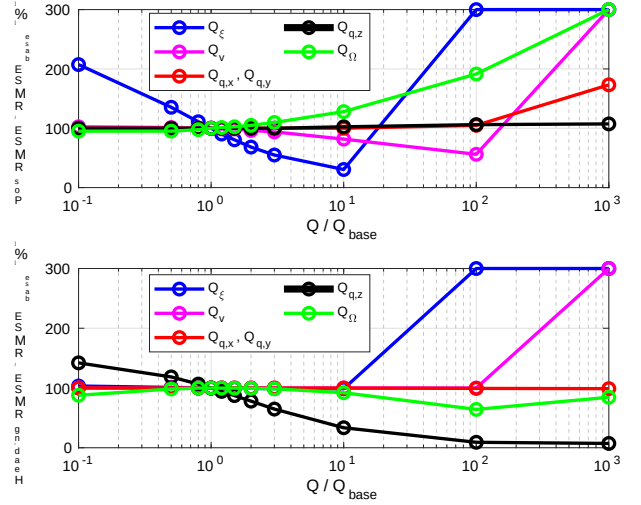


图10: NMPC方法在不同控制参数下的位置 (上) 和航向 (下) 跟踪RMSE。纵轴表示RMSE与基线参数设置下RMSE的比值。横轴表示控制参数与基线参数的比值。那些坠机飞行的RMSE被限制在 $RMSE_{base}$ 的300%

还会对位置跟踪性能产生对抗性影响, 因为它削弱了位置权重的作用。关于航向控制, 正如预期的那样, 增加 $Q_{q,z}$ 可以提高航向跟踪性能, 同时对位置跟踪几乎没有影响。

2) *Differential Flatness Based Control*: DFBC 的控制器增益包括位置增益、姿态增益以及用于控制分配的权重矩阵。增益的选择是为了使位置误差 $\xi_e = \xi - \xi_r$ 成为二阶系统:

$$\ddot{\xi}_e + 2\zeta_\xi \omega_{n,\xi} \dot{\xi}_e + \omega_{n,\xi}^2 \xi_e = 0 \quad (40)$$

其中 ζ_ξ 和 $\omega_{n,\xi}$ 分别是阻尼比和固有频率。然后将 (13) 代入 (40), 并用 $\ddot{\xi}$ (替换 $\ddot{\xi}_d$, 假设完美的内环姿态控制), 我们可以按如下方法计算位置控制增益:

$$K_\xi = \text{diag}(\omega_{n,\xi}^2, \omega_{n,\xi}^2, \omega_{n,\xi}^2), K_v = 2\zeta_\xi \sqrt{K_\xi}. \quad (41)$$

同样, 姿态增益可以设计为:

$$k_{q,x} = k_{q,y} = \omega_{n,xy}^2, k_{q,z} = \omega_{n,z}^2, \quad (42)$$

$$K_\Omega = 2\zeta_q \text{diag}(\sqrt{k_{q,x}}, \sqrt{k_{q,y}}, \sqrt{k_{q,z}}). \quad (43)$$

其中 ζ_q 是阻尼比, $\omega_{n,xy}$ 和 $\omega_{n,z}$ 分别是简化姿态误差和偏航误差的期望固有频率。

图11展示了跟踪误差与闭环自然频率之间的关系。时间常数和跟踪误差均通过表I中给出的基线增益进行了归一化。较高的频率表示较高的控制增益, 从而导致较小的跟踪误差。然而, 高增益也容易受到系统延迟的影响, 进而引起不稳定性。

权重矩阵 W 是基于模拟中的调试选择的, 而无需复杂计算。其目的是保留

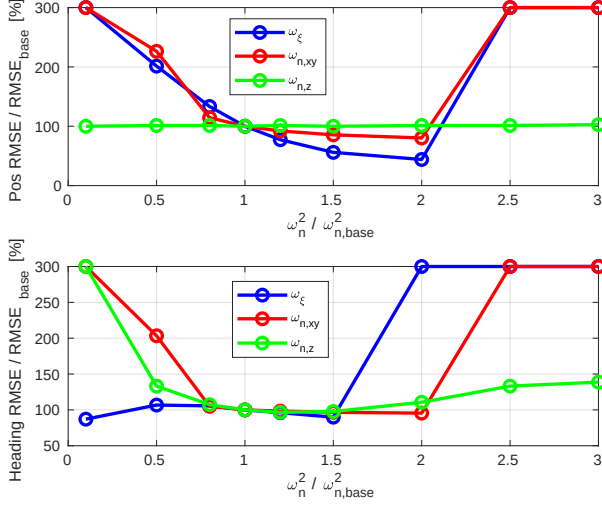


Fig. 11: Position (top) and heading (bottom) tracking RMSE of the DFBC method in different control gain settings. The vertical axis represents the ratio between RMSE and the RMSE from a baseline gain setup. The horizontal axis represents the ratio between the controller gains and the baseline gains. RMSE of the crashed flights are clamped at 300% of $RMSE_{base}$.

more priority to the tilt control over collective thrust and yaw. An example of inappropriate selection of W is an identical matrix, which makes the QP-based allocation identical to a direct inversion. The performance in terms of crash rate has been compared in Fig. 7 necessity of using a well-tuned W in tracking dynamically *infeasible* trajectories.

VII. REAL-WORLD EXPERIMENTS

We conduct the experiments on a quadrotor in an instrumented tracking arena. A set of aggressive trajectories are executed to compare the closed-loop tracking performance of NMPC and DFBC in the presence of joint effects such as model mismatch, state estimation error, and system latency. The real-world experiments also highlight the contributions of the INDI low-level controller, the aerodynamic force model, to both NMPC and DFBC methods.

Experiments are conducted in tracking different reference trajectories, ranging from simple loop trajectories to FPV drone racing tracks (see Table. VI). Trajectories with the same 3D shape but different velocities and accelerations are also tested. The 3D paths of these trajectories are illustrated in Fig. 12.

We evaluated NMPC and DFBC, with and without the INDI inner-loop controller. NMPC with a PID low-level controller is also tested for comparison, implemented to track time-optimal trajectory in [4]. In the following, these methods consider aerodynamic drag models by default, unless further mentioned. The position tracking RMSE and heading tracking RMSE are listed in Table. VI and Table. VII.

A. Contribution of the INDI and Drag Model

Table VIII compares the average RMSE of both methods and shows the effect of the INDI inner-loop controller, and

the effect of the aerodynamic drag model. For NMPC and DFBC, neglecting aerodynamic drag would increase position tracking error by 144% percent and 122%, respectively.

In contrast to the aerodynamic drag model, removing the INDI low-level controller influences more on the tracking accuracy. For NMPC, we see more than 364% and 115% increase in position and heading RMSE if INDI is not used. For DFBC, removing INDI leads to a more severe consequence: more than 705% and 492% of position and heading RMSE increase. In fact, DFBC without INDI cannot successfully track some of these trajectories without crashing the drone. These results indicate that, an adaptive/robust inner-loop controller plays a more important role than the drag model in accurately tracking aggressive trajectories.

We also make comparisons against the control method used in [4], which uses PID as the inner-loop controller of NMPC. This comparison is made in tracking a time-optimal trajectory generated off-line using the algorithm proposed in [4]. In this task, the quadrotor needs to fly through multiple gates in a predefined order. Fig. 13 demonstrates the tracking results, including the 3D path and the position error. Clearly, both NMPC and DFBC augmented by INDI significantly outperform this NMPC with PID inner-loop controller. Comparative results on other reference trajectories can be found in Table VI. On average, the proposed NMPC with the INDI approach shows a position RMSE of 0.102 m, which is 70% lower than the position RMSE of NMPC with PID low-level controller (0.343 m).

B. Tracking Dynamically Infeasible Trajectories

In these real-world experiments, we also track three dynamically infeasible trajectories that exceed the maximum thrust of the tested quadrotor. Comparative results of NMPC and DFBC are given in Table VII. Clearly, NMPC shows significantly higher tracking accuracy than DFBC when tracking these dynamically infeasible trajectories, which is consistent with the simulation results in Sec. VI.

Fig. 14 presents the 3D path tracking the *Loop C* reference trajectory. Tracking such a reference loop trajectory requires a higher collective thrust than the maximum thrust of the tested quadrotor. Hence the quadrotor cannot follow the reference trajectory and both control methods make the quadrotor take the shortcut to fly inside the reference trajectory. While DFBC results in a chaotic trajectory, NMPC makes the drone fly a much more regular loop trajectory with a smaller radius than the reference, thus requiring less collective thrust. The difference on the side-view is more distinctive: NMPC has much higher altitude tracking accuracy than the DFBC method.

C. Computational Time

Fig. 15 and 16 respectively show the CPU time of DFBC and NMPC with INDI as the inner-loop controller while tracking the *Race Track C* trajectory. The computation time of DFBC is generally faster than 0.025 ms, which is significantly faster compared with NMPC that takes around 3 ms. In addition, both QP and INDI modules spend less than 0.01 ms CPU time.

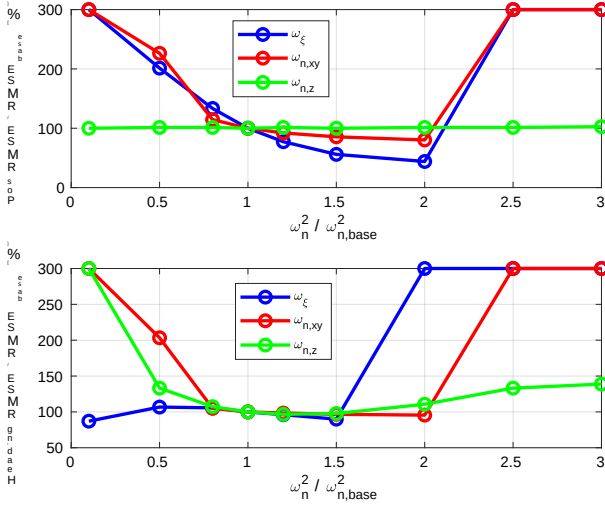


图11: 在不同控制增益设置下, DFBC方法的位置(上)和航向(下)跟踪均方根误差(RMSE)。纵轴表示RMSE与基线增益设置下RMSE的比值。横轴表示控制器增益与基线增益的比值。坠机航班的RMSE被限制在 $RMSE_{base}$ 的300%以内。

更多优先考虑倾斜控制而不是集体推力和偏航。 $\mathbf{W}$ 不适当地选择一个例子是相同的矩阵, 这会使基于 QP 的分配与直接反演相同。如图 7 所示, 在碰撞率方面的性能进行了比较, 说明在跟踪动态 *infeasible* 轨迹时使用调校良好的 $\mathbf{W}$ 的必要性。

七、现实世界实验

我们在一个配备仪器的跟踪场地对四旋翼进行了实验。执行了一系列激进的轨迹, 以比较在存在例如模型不匹配、状态估计误差和系统延迟等关节效应时, NMPC 和 DFBC 的闭环跟踪性能。真实世界的实验还突出了 INDI 低级控制器和气动阻力模型对 NMPC 和 DFBC 方法的贡献。

进行实验以跟踪不同的参考轨迹, 范围从简单的环形轨迹到FPV无人机竞速轨道(见表 VI)。还测试了具有相同三维形状但不同速度和加速度的轨迹。这些轨迹的三维路径如图 12 所示。

我们评估了 NMPC 和 DFBC, 有和没有 INDI 内环控制器的情况。为了对比, 还测试了配备 PID 低级控制器的 NMPC, 其实现用于跟踪 [4] 中的时间最优轨迹。以下情况下, 这些方法默认考虑空气动力学阻力模型, 除非另有说明。位置跟踪 RMSE 和航向跟踪 RMSE 列于表 VI 和表 VII。

A. Contribution of the INDI and Drag Model

表 VIII 比较了两种方法的平均 RMSE, 并显示了 INDI 内环控制器的效果, 且

空气动力阻力模型的影响。对于 NMPC 和 DFBC, 忽略空气动力阻力将分别使位置跟踪误差增加 144% 和 122%。

与气动阻力模型相比, 移除 INDI 低级控制器对跟踪精度的影响更大。对于 NMPC, 如果不使用 INDI, 位置和航向的均方根误差(RMSE)分别增加超过 364% 和 115%。对于 DFBC, 移除 INDI 导致的后果更加严重: 位置和航向 RMSE 分别增加超过 705% 和 492%。实际上, 没有 INDI 的 DFBC 无法在不撞毁无人机的情况下成功跟踪这些轨迹。这些结果表明, 自适应/鲁棒内环控制器在精确跟踪激进轨迹方面比阻力模型起的作用更重要。

我们还与文献[4]中使用的控制方法进行了比较, 该方法使用PID作为NMPC的内环控制器。该比较是针对使用文献[4]提出的算法离线生成的时间最优轨迹的跟踪进行的。在此任务中, 四旋翼需要按预定顺序飞过多个门。图13展示了跟踪结果, 包括三维路径和位置误差。显然, 通过INDI增强的NMPC和DFBC都明显优于带PID内环控制器的NMPC。其他参考轨迹的比较结果可以在表VI中找到。平均而言, 所提出的结合INDI方法的NMPC显示出0.102 m的位置RMSE, 比使用PID低级控制器的NMPC位置RMSE (0.343 m) 低70%。

B. Tracking Dynamically Infeasible Trajectories

在这些现实世界的实验中, 我们还跟踪了三条超过所测试四旋翼最大推力的动态不可行轨迹。NMPC 和 DFBC 的比较结果如表 VII 所示。显然, 在跟踪这些动态不可行轨迹时, NMPC 显示出明显高于 DFBC 的跟踪精度, 这与第 VI 节的仿真结果一致。

图14展示了3D路径跟踪Loop C参考轨迹。跟踪这样的参考环路轨迹需要比所测试四旋翼最大推力更大的总体推力。因此四旋翼无法跟随参考轨迹, 且两种控制方法都让四旋翼采取捷径飞入参考轨迹内部。虽然DFBC导致轨迹混乱, NMPC使无人机飞行一个比参考轨迹半径更小的、更规则的环路轨迹, 从而需要更少的总体推力。侧视图上的差异更为明显: NMPC的高度跟踪精度远高于DFBC方法。

C. Computational Time

图15和图16分别展示了在跟踪Race Track C轨迹时, 使用INDI作为内环控制器的DFBC和NMPC的CPU时间。DFBC的计算时间通常快于0.025毫秒, 相比之下, NMPC约需3毫秒, DFBC明显更快。此外, QP和INDI模块的CPU时间均少于0.01毫秒。

TABLE VI: Tracking performance of different methods in real-world experiments

References			Position Tracking RMSE [m]							Heading Tracking RMSE [deg]						
Type	$V_{\max}$ [m/s]	$a_{\max}$ [m/s ²]	NMPC+PID [4]	NMPC	NMPC+INDI	NMPC+INDI (w/o drag model)	DFBC	DFBC+INDI	DFBC+INDI (w/o drag model)	NMPC+PID [4]	NMPC	NMPC+INDI	NMPC+ IND I (w/o drag model)	DFBC	DFBC+ INDI	DFBC+ INDI (w/o drag model)
Loop A	10.2	20.7	0.280	0.548	0.068	0.215	0.897	0.073	0.242	6.801	11.274	8.450	8.972	73.842	4.411	9.395
Loop B	13.4	35.9	0.369	0.930	0.117	0.261	-	0.125	0.252	11.618	22.193	9.714	10.478	-	10.496	11.489
Slant Loop A	8.2	12.6	0.241	0.458	0.077	0.156	0.535	0.109	0.163	6.951	19.795	5.987	6.870	17.755	7.521	8.032
Slant Loop B	12.9	30.8	0.447	0.777	0.116	0.217	-	0.138	0.305	7.729	18.382	8.282	8.816	-	9.171	10.047
Vertical Loop A	7.7	14.6	0.197	0.206	0.069	0.172	0.181	0.082	0.127	1.558	5.131	1.108	1.274	30.901	1.151	1.530
Vertical Loop B	11.5	32.9	0.482	0.350	0.126	0.314	-	0.177	0.196	19.000	6.019	2.415	1.900	-	15.495	8.632
Oscillate A	10.9	20.7	0.327	0.212	0.058	0.259	0.310	0.164	0.210	1.641	7.369	2.029	1.960	50.210	3.977	1.911
Oscillate B	14.0	36.5	0.375	0.290	0.125	0.377	-	0.129	0.294	2.708	9.228	2.386	2.902	-	2.763	3.821
Hairpin A	7.7	14.6	0.185	0.352	0.024	0.146	0.350	0.049	0.124	4.733	7.100	3.745	4.255	38.980	5.913	5.873
Hairpin B	11.5	32.9	0.390	0.479	0.099	0.274	1.797	0.117	0.166	7.965	15.748	7.343	8.017	68.139	10.163	8.004
Lemniscate A	9.5	13.6	0.225	0.324	0.056	0.184	0.423	0.068	0.216	3.576	9.074	3.689	4.109	37.235	2.605	2.243
Lemniscate B	14.2	36.0	0.314	0.564	0.130	0.290	-	0.128	0.287	7.335	15.952	6.280	6.659	-	4.986	5.403
Split-S	14.2	25.5	0.292	0.352	0.096	0.262	-	0.087	0.239	9.411	13.253	6.409	5.259	-	17.888	18.929
Racing Track A	11.9	22.1	0.280	0.424	0.108	0.212	0.552	0.102	0.240	6.806	10.434	6.566	7.706	37.731	4.336	5.390
Racing Track B	16.8	28.5	0.369	0.546	0.130	0.320	3.797	0.152	0.359	11.930	14.672	9.318	8.805	57.810	8.249	7.719
Racing Track C	19.4	37.3	0.712	0.758	0.238	0.329	-	0.259	0.480	11.235	19.936	11.754	9.876	-	14.695	15.498
Mean			0.343	0.473	0.102	0.249	0.982	0.122	0.244	7.562	12.848	5.967	6.116	45.845	7.739	7.745
Standard deviation			0.130	0.207	0.067	0.048	1.161	0.164	0.051	4.499	5.434	3.054	3.136	18.175	4.768	4.964

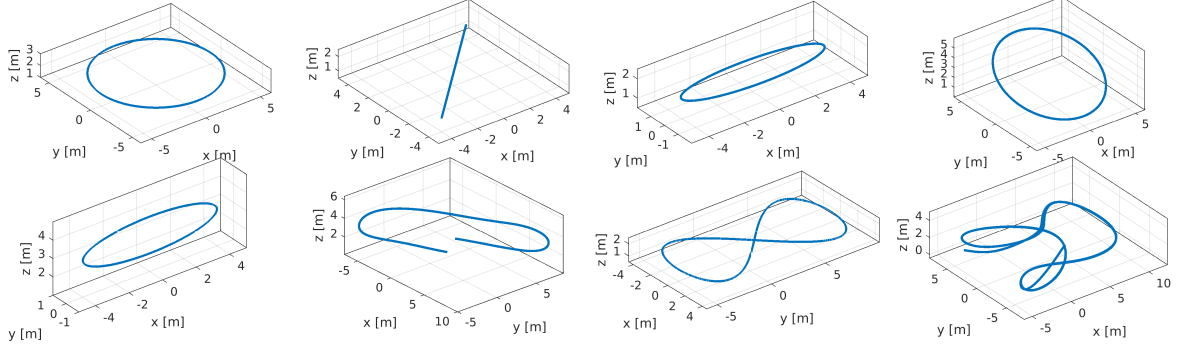


Fig. 12: Tested trajectories in real-world experiments. From top-left to bottom right, they are: Loop, Oscillate, Hairpin, Slant-loop, Vertical-loop, Split-S, Lemniscate, Racing-trajectory.

TABLE VII: Performance in tracking three dynamically infeasible trajectories in real-world experiments. Position and heading RMSE comparisons between NMPC and DFBC with INDI inner-loop.

Reference	$V_{\max}$ [m]	$a_{\max}$ [m/s ²]	Position RMSE [m]		Heading RMSE [deg]	
			NMPC +INDI	DFBC +INDI	NMPC +INDI	DFBC +INDI
Vertical Loop C	12.1	48.5	0.318	0.919	2.469	25.608
Loop C	15.7	49.3	0.762	1.829	13.439	25.876
Lemniscate C	19.0	54.5	0.570	1.237	11.680	53.262
Average RMSE			0.550	1.328	9.196	34.915

TABLE VIII: Position and heading tracking RMSE of NMPC and DFBC methods. Adding INDI as the inner-loop improves the performance significantly for both methods.

	Position RMSE [m]		Heading RMSE [deg]	
NMPC+INDI	0.102	(↑ 0%)	5.967	(↑ 0%)
NMPC+INDI (w/o drag model)	0.249	(↑ 144%)	6.116	(↑ 3%)
NMPC	0.473	(↑ 364%)	12.848	(↑ 115%)
DFBC+INDI	0.122	(↑ 0%)	7.739	(↑ 0%)
DFBC+INDI (w/o drag model)	0.271	(↑ 122%)	7.745	(↑ 0.07%)
DFBC	0.982	(↑ 705%)	45.845	(↑ 492%)

Fig. 17 compares the average computational time of gen-

erating each control commands of NMPC and DFBC, with INDI using all the flight data. Each data point in the box plot represents the average computing time of a single flight. As is expected, NMPC requires significantly longer time to generate a single control command (2.7 ms) compared with DFBC (0.020 ms). Be that as it may, both methods can run onboard at sufficiently high frequency (≥ 100 Hz) to achieve accurate tracking of agile trajectories.

VIII. DISCUSSION

According to the simulation and real-world flight results, we conclude that both NMPC and DFBC, with an INDI inner-loop controller, can track highly aggressive trajectories with similar accuracy as long as the reference trajectories are dynamically *feasible*.

The advantage of NMPC appears when tracking dynamically *infeasible* trajectories that violate the rotor thrust constraints. Even though DFBC also uses the constrained-quadratic programming for control allocation, it only considers a single reference point; thus, the actions taken are too short-sighted to avoid future violations of these constraints. By

表 VI: 不同方法在实际实验中的跟踪性能

References			Position Tracking RMSE [m]							Heading Tracking RMSE [deg]						
Type	V_m	a_m	$\begin{smallmatrix} D \\ P+C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} D \\ N+C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} D \\ N+C \\ P+W \end{smallmatrix}$ M N	$\begin{smallmatrix} C \\ B \\ F \\ D \end{smallmatrix}$	$\begin{smallmatrix} D \\ N+C \\ B \\ F \\ D \end{smallmatrix}$	$\begin{smallmatrix} D \\ N+C \\ B \\ F \\ D+W \end{smallmatrix}$ M N	$\begin{smallmatrix} D \\ P+C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} D \\ N+C \\ P \end{smallmatrix}$ M N	$\begin{smallmatrix} D \\ N+C \\ P+W \end{smallmatrix}$ M N	$\begin{smallmatrix} C \\ B \\ F \\ D \end{smallmatrix}$	$\begin{smallmatrix} D \\ N+C \\ B \\ F \\ D \end{smallmatrix}$	$\begin{smallmatrix} D \\ N+C \\ B \\ F \\ D+W \end{smallmatrix}$ M N
Loop A	10.2	20.7	0.280	0.548	0.068	0.215	0.897	0.073	0.242	6.801	11.274	8.450	8.972	73.842	4.411	9.395
Loop B	13.4	35.9	0.369	0.930	0.117	0.261	-	0.125	0.252	11.618	22.193	9.714	10.478	-	10.496	11.489
Slant Loop A	8.2	12.6	0.241	0.458	0.077	0.156	0.535	0.109	0.163	6.951	19.795	5.987	6.870	17.755	7.521	8.032
Slant Loop B	12.9	30.8	0.447	0.777	0.116	0.217	-	0.138	0.305	7.729	18.382	8.282	8.816	-	9.171	10.047
Vertical Loop A	7.7	14.6	0.197	0.206	0.069	0.172	0.181	0.082	0.127	1.558	5.131	1.108	1.274	30.901	1.151	1.530
Vertical Loop B	11.5	32.9	0.482	0.350	0.126	0.314	-	0.177	0.196	19.000	6.019	2.415	1.900	-	15.495	8.632
Oscillate A	10.9	20.7	0.327	0.212	0.058	0.259	0.310	0.164	0.210	1.641	7.369	2.029	1.960	50.210	3.977	1.911
Oscillate B	14.0	36.5	0.375	0.290	0.125	0.377	-	0.129	0.294	2.708	9.228	2.386	2.902	-	2.763	3.821
Hairpin A	7.7	14.6	0.185	0.352	0.024	0.146	0.350	0.049	0.124	4.733	7.100	3.745	4.255	38.980	5.913	5.873
Hairpin B	11.5	32.9	0.390	0.479	0.099	0.274	1.797	0.117	0.166	7.965	15.748	7.343	8.017	68.139	10.163	8.004
Lemniscate A	9.5	13.6	0.225	0.324	0.056	0.184	0.423	0.068	0.216	3.576	9.074	3.689	4.109	37.235	2.605	2.243
Lemniscate B	14.2	36.0	0.314	0.564	0.130	0.290	-	0.128	0.287	7.335	15.952	6.280	6.659	-	4.986	5.403
Split-S	14.2	25.5	0.292	0.352	0.096	0.262	-	0.087	0.239	9.411	13.253	6.409	5.259	-	17.888	18.929
Racing Track A	11.9	22.1	0.280	0.424	0.108	0.212	0.552	0.102	0.240	6.806	10.434	6.566	7.706	37.731	4.336	5.390
Racing Track B	16.8	28.5	0.369	0.546	0.130	0.320	3.797	0.152	0.359	11.930	14.672	9.318	8.805	57.810	8.249	7.719
Racing Track C	19.4	37.3	0.712	0.758	0.238	0.329	-	0.259	0.480	11.235	19.936	11.754	9.876	-	14.695	15.498
Mean			0.343	0.473	0.102	0.249	0.982	0.122	0.244	7.562	12.848	5.967	6.116	45.845	7.739	7.745
Standard deviation			0.130	0.207	0.067	0.048	1.161	0.164	0.051	4.499	5.434	3.054	3.136	18.175	4.768	4.964

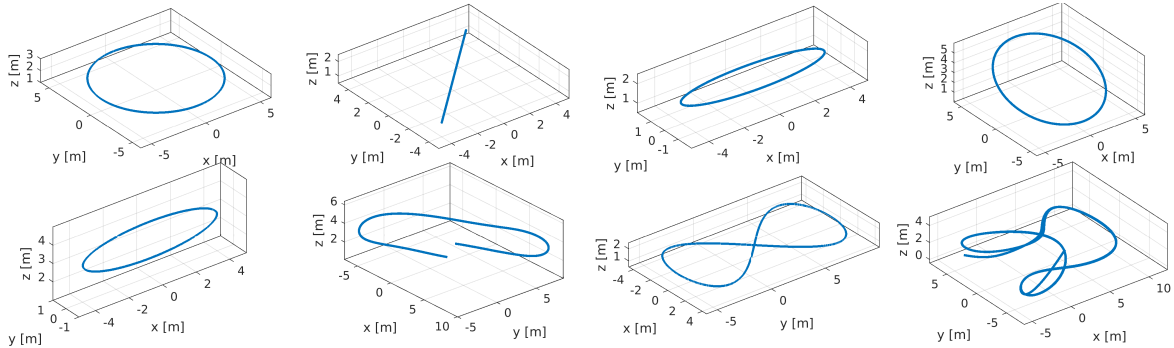


图 12: 在现实实验中测试的轨迹。从左上到右下, 它们是: 环路、振荡、发夹、斜环、垂直-循环、分裂-S, 勒姆尼茨曲线, 赛车轨迹。

表 VII: 在实际实验中跟踪三条动态不可行轨迹的性能。NMPC 与基于 INDI 内环的 DFBC 在位置和航向均方根误差的比较。

Reference	V_{max} [m/s]	a_{max} [m/s ²]	Position RMSE [m]		Heading RMSE [deg]	
			NMPC +INDI	DFBC +INDI	NMPC +INDI	DFBC +INDI
Vertical Loop C	12.1	48.5	0.318	0.919	2.469	25.608
Loop C	15.7	49.3	0.762	1.829	13.439	25.876
Lemniscate C	19.0	54.5	0.570	1.237	11.680	53.262
Average RMSE			0.550	1.328	9.196	34.915

表 VIII: NMPC 和 DFBC 方法的位姿和航向跟踪均方根误差。将 INDI 作为内环引入显著提高了两种方法的性能。

	Position RMSE [m]		Heading RMSE [deg]	
NMPC+INDI	0.102	(↑ 0%)	5.967	(↑ 0%)
NMPC+INDI (w/o drag model)	0.249	(↑ 144%)	6.116	(↑ 3%)
NMPC	0.473	(↑ 364%)	12.848	(↑ 115%)
DFBC+INDI	0.122	(↑ 0%)	7.739	(↑ 0%)
DFBC+INDI (w/o drag model)	0.271	(↑ 122%)	7.745	(↑ 0.07%)
DFBC	0.982	(↑ 705%)	45.845	(↑ 492%)

图17比较了gen的平均计算时间

对 NMPC 和 DFBC 的每个控制指令进行操作, INDI 则使用所有飞行数据。箱线图上的每个数据点代表一次飞行的平均计算时间。正如预期的那样, NMPC 生成单个控制指令所需的时间 (2.7 毫秒) 明显长于 DFBC (0.020 毫秒)。尽管如此, 两种方法都可以在足够高的频率 (约 100 Hz) 在机载运行, 以实现敏捷轨迹的精确跟踪。

八、讨论

根据仿真和实际飞行结果, 我们得出结论, 只要参考轨迹在动态上是 *feasible* 的, NMPC 和 DFBC 在配备 INDI 内环控制器的情况下, 都可以以类似的精度跟踪高度激进的轨迹。

当跟踪违反转子推力约束的动态 *infeasible* 轨迹时, NMPC 的优势就显现出来。尽管 DFBC 也使用受约束的二次规划进行控制分配, 但它只考虑单个参考点; 因此, 其采取的行动过于短视, 无法避免未来违反这些约束。由

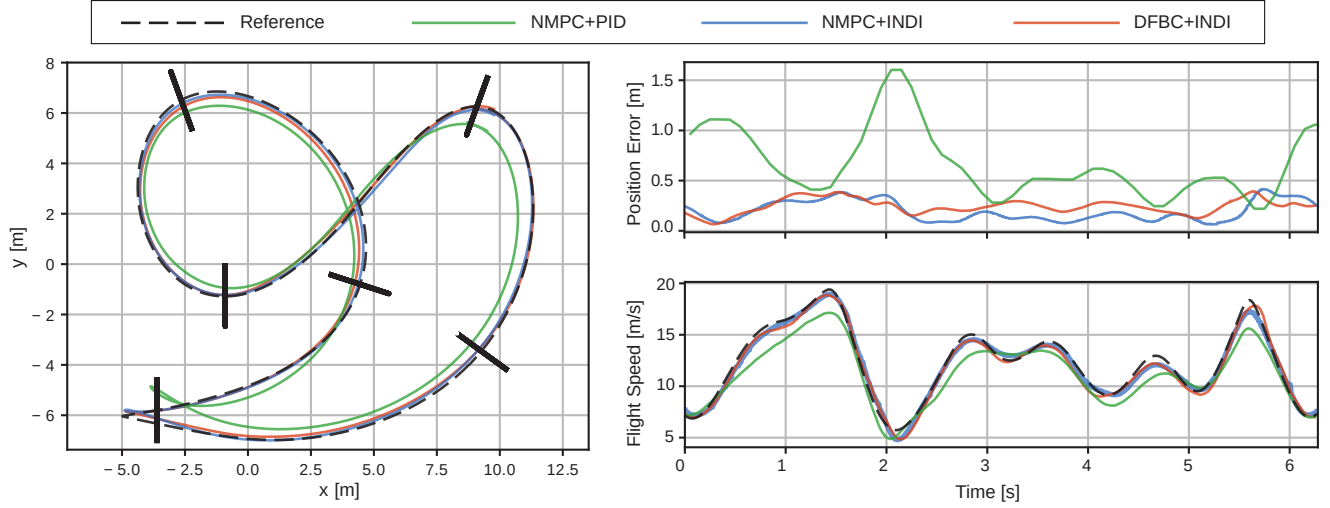


Fig. 13: Tracking performance of the *Race Track C* with a maximum speed up to 20 m/s. NMPC and DFBC with INDI inner-loop show similar tracking performance. Both outperform the state-of-the-art NMPC with a PID inner-loop controller [4]

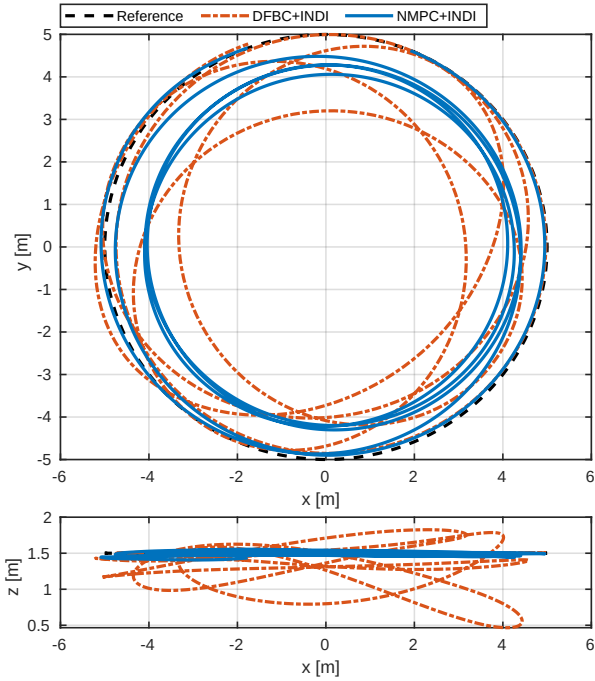


Fig. 14: Top and side view of the trajectory tracking a dynamically infeasible trajectory (black-dash) using NMPC (blue-solid) and DFBC method (red-dash-dot).

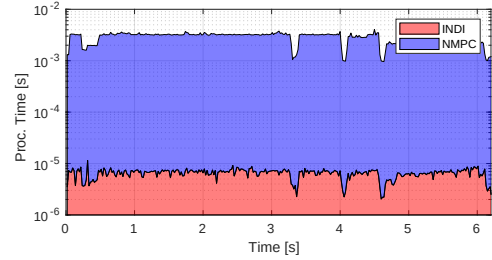


Fig. 16: CPU time of NMPC+INDI while tracking the *Race Track C* trajectory. Note that the vertical axis is presented in the log scale.

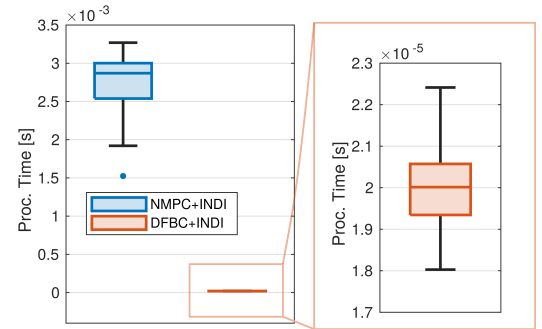


Fig. 17: Processing time to generate each control command of NMPC and DFBC methods.

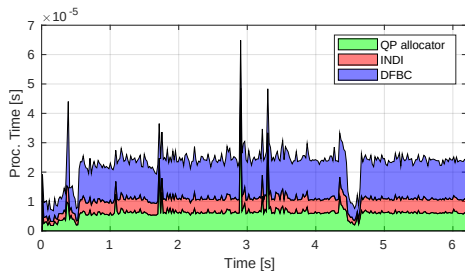


Fig. 15: CPU time of DFBC+INDI while tracking the *Race Track C* trajectory.

contrast, NMPC tracks the *infeasible* trajectory using future predictions, including multiple reference points that minimize the tracking error and respect single rotor constraints. Such difference helps NMPC outperform DFBC by 48% and 62% on position and heading accuracy, respectively, if the reference trajectories are dynamically *infeasible*.

NMPC also demonstrates higher robustness against the rotational model mismatch. In real-world experiments, we also performed tests without INDI low-level controller for comparisons. As such, both methods suffered from model uncertainties on the rotational dynamics. In this condition,

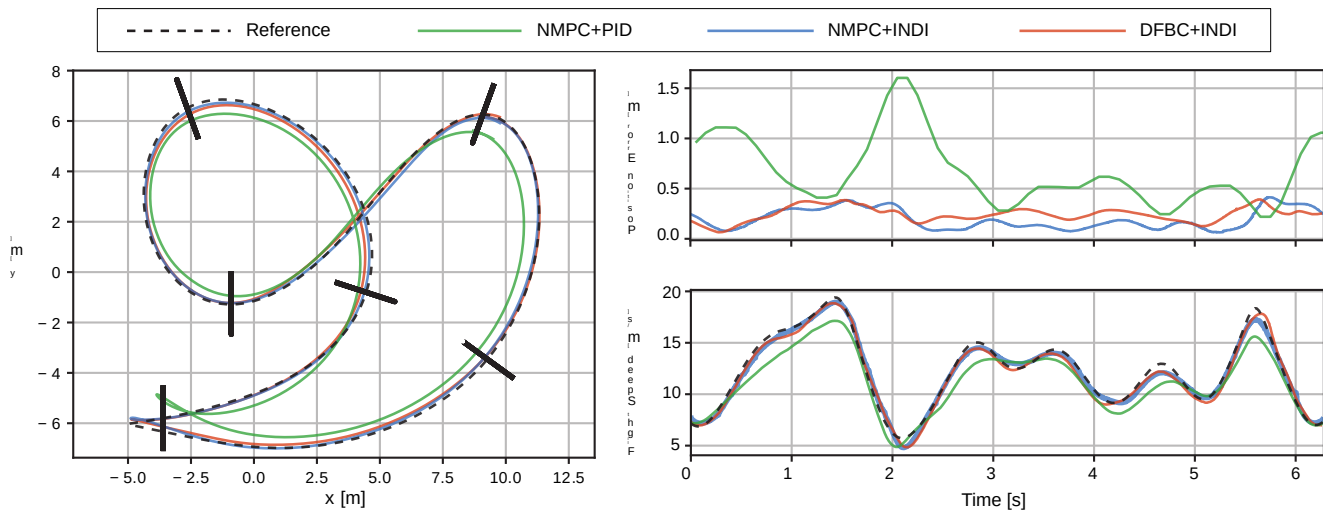


图 13: 跟踪Race Track C的性能, 最大速度可达20米/秒。使用INDI内环的NMPC和DFBC显示相似ar跟踪性能。两者都优于采用PID内环控制器的最先进NMPC [4]

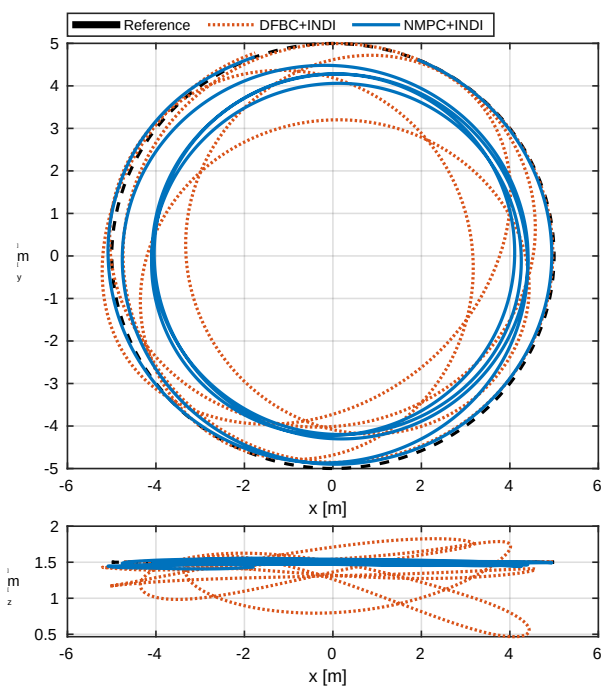


图14: 采用NMPC (蓝色实线) 和DFBC方法 (红色点划线) 跟踪动态不可行轨迹 (黑色虚线) 的轨迹的顶部视图和侧视图。

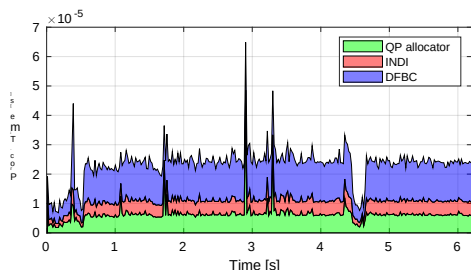


图15: DFBC+INDI在跟踪Race Track C轨迹时的CPU时间。

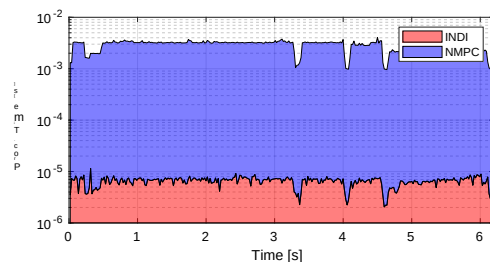


图16: NMPC+INDI在跟踪Race Track C轨迹时的CPU时间。注意, 纵轴以对数刻度显示。

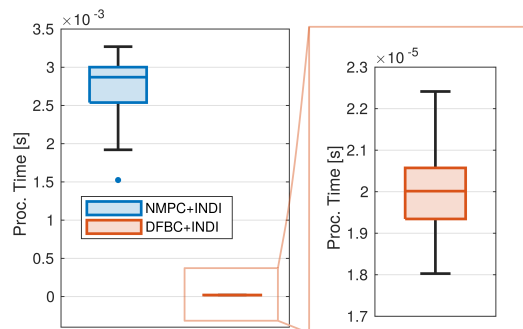


图17: 生成NMPC和DFBC方法每个控制指令的处理时间。

相比之下, NMPC 使用未来的预测跟踪 *infeasible* 轨迹, 包括多个最小化跟踪误差并满足单转子约束的参考点。如果参考轨迹是动态 *infeasible* 的, 这种差异使 NMPC 在位置和航向精度上分别比 DFBC 高出 48% 和 62%。

NMPC 在应对旋转模型不匹配时也表现出更高的鲁棒性。在实际实验中, 我们还进行了没有 INDI 低级控制器的测试以作比较。因此, 两种方法都受到旋转动力学模型不确定性的影响。在这种情况下,

NMPC was still able to track all the trajectories, whereas DFBC experienced several crashes and had much higher tracking RMSE against NMPC.

However, NMPC also has limitations. For example, the biggest disadvantage of NMPC is that it requires significantly higher computational resources. On our tested hardware, the average solving time of the nonlinear NMPC is around 2.7 ms, while DFBC needs only 0.020 ms, which is around 100 times faster. This renders it impractical to run NMPC on some miniature aerial vehicles with a limited computational budget, such as Crazyflie [44]. By contrast, we can deploy DFBC on light-weighted drones with low-end processors and conduct agile flights at speeds over 20 m/s, as long as the platform has enough thrust-to-weight ratio.

Another disadvantage of NMPC is that it potentially suffers from numerical convergence issues. Unlike DFBC, which has proof of stability or convergence of each sub-module, the nonlinear NMPC used in this comparison relies on the numerical convergence of the nonlinear optimization algorithm. Unfortunately, rigorous proof of its convergence is still an open question. In fact, nonlinear NMPC tends to fail in converging when the current position is too far from the reference, either caused by large external force disturbances or an error in the thrust-to-weight ratio model. For example, our robustness study shows a 10% higher crash rate of NMPC than DFBC when the real mass is 30% higher than the model. In addition, we also observe that NMPC is more prone to fail in converging than DFBC in the presence of large system latency.

In summary, NMPC is not superior to DFBC in all scenarios. If the reference trajectories are dynamically *feasible*, DFBC can achieve similar tracking performance while consuming only 1 percent of the control resource that NMPC requires. However, it is difficult to justify the feasibility of a trajectory near the physical limitations of the platform because of the model uncertainties such as aerodynamic effects.

Hence, NMPC excels at exploiting the full capability of an autonomous drone in a safer and more efficient manner, due to its advantage in handling the infeasibility of reference trajectories. Future work may further hybridize NMPC with differential flatness property to reduce the NMPC computational cost while retaining its advantage in handling dynamically infeasible trajectories. The reason why NMPC failed in converging also needs further investigation in order to deal with larger amount of model uncertainties.

IX. CONCLUSION

This work systematically compares NMPC and DFBC, two state-of-the-art quadrotor controllers for tracking agile trajectories. Simulation and real-world experiments are extensively performed to evaluate the performance of both methods in terms of accuracy, robustness, and computational efficiency. We report the advantages and disadvantages of both methods according to the results. This work also evaluates the effect of INDI inner-loop control on both methods. Real-world flight results at up to 20 m/s demonstrate the necessity of applying an INDI inner-loop on both NMPC and DFBC approaches. The effect of the high-speed aerodynamic drag model is also

evaluated, which is found less influential compared to the inner-loop controller.

REFERENCES

- [1] G. Loianno and D. Scaramuzza, “Special issue on future challenges and opportunities in vision-based drone navigation,” *Journal of Field Robotics*, vol. 37, no. 4, pp. 495–496, 2020.
- [2] S. Rajendran and S. Srinivas, “Air taxi service for urban mobility: A critical review of recent developments, future challenges, and opportunities,” *Transportation research part E: logistics and transportation review*, vol. 143, p. 102090, 2020.
- [3] D. Bicego, J. Mazzetto, R. Carli, M. Farina, and A. Franchi, “Nonlinear model predictive control with enhanced actuator model for multi-rotor aerial vehicles with generic designs,” *Journal of Intelligent & Robotic Systems*, vol. 100, no. 3, pp. 1213–1247, 2020.
- [4] P. Foehn, A. Romero, and D. Scaramuzza, “Time-optimal planning for quadrotor waypoint flight,” *Science Robotics*, vol. 6, no. 56, p. eabh1221, 2021.
- [5] G. Torrente, E. Kaufmann, P. Föhn, and D. Scaramuzza, “Data-driven mpc for quadrotors,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3769–3776, 2021.
- [6] M. Kamel, K. Alexis, M. Achtelik, and R. Siegwart, “Fast nonlinear model predictive control for multicopter attitude tracking on so(3),” in *2015 IEEE Conference on Control Applications (CCA)*, pp. 1160–1166, IEEE, 2015.
- [7] M. Kamel, T. Stastny, K. Alexis, and R. Siegwart, “Model predictive control for trajectory tracking of unmanned aerial vehicles using robot operating system,” in *Robot operating system (ROS)*, pp. 3–39, Springer, 2017.
- [8] A. Murilo and R. V. Lopes, “Unified nmpc framework for attitude and position control for a vtol uav,” *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, vol. 233, no. 7, pp. 889–903, 2019.
- [9] E. Small, P. Sopasakis, E. Fresk, P. Patrinos, and G. Nikolakopoulos, “Aerial navigation in obstructed environments with embedded nonlinear model predictive control,” in *2019 18th European Control Conference (ECC)*, pp. 3556–3563, IEEE, 2019.
- [10] H. Nguyen, M. Kamel, K. Alexis, and R. Siegwart, “Model predictive control for micro aerial vehicles: A survey,” in *2021 European Control Conference (ECC)*, pp. 1556–1563, IEEE, 2021.
- [11] A. Romero, P. Foehn, and D. Scaramuzza, “Model predictive contouring control for time-optimal quadrotor flight,” *IEEE Transactions on Robotics*, 2022.
- [12] M. Faessler, A. Franchi, and D. Scaramuzza, “Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories,” *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, 2017.
- [13] E. Tal and S. Karaman, “Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness,” *IEEE Trans-*

NMPC 仍然能够追踪所有轨迹, 而 DFBC 出现了多次崩溃, 并且相对于 NMPC 具有更高的追踪均方根误差 (RMSE)。

然而, NMPC 也有其局限性。例如, NMPC 最大的缺点是它需要显著更高的计算资源。在我们测试的硬件上, 非线性 NMPC 的平均求解时间约为 2.7 毫秒, 而 DFBC 仅需 0.020 毫秒, 大约快 100 倍。这使得在一些计算能力有限的小型飞行器上运行 NMPC 不切实际, 例如 Crazyflie [44]。相比之下, 只要平台具有足够的推重比, 我们可以在轻量级的低端处理器无人机上部署 DFBC, 并进行速度超过 20 m/s 的敏捷飞行。

NMPC 的另一个缺点是它可能会遇到数值收敛问题。与 DFBC 不同, DFBC 对每个子模块的稳定性或收敛性都有证明, 而本次比较中使用的非线性 NMPC 则依赖于非线性优化算法的数值收敛。不幸的是, 其收敛性的严格证明仍是一个悬而未决的问题。事实上, 当当前位置距离参考位置过远时, 非线性 NMPC 往往难以收敛, 这可能是由于较大的外部力扰动或推重比模型存在误差造成的。例如, 我们的稳健性研究显示, 当实际质量比模型高 30% 时, NMPC 的碰撞率比 DFBC 高出 10%。此外, 我们还观察到, 在存在较大系统延迟的情况下, NMPC 比 DFBC 更容易收敛失败。

总之, NMPC 并非在所有场景下都优于 DFBC。如果参考轨迹是动态 *feasible* 的, DFBC 可以实现类似的跟踪性能, 同时只消耗 NMPC 所需控制资源的 1%。然而, 由于模型不确定性 (如空气动力学效应), 很难证明接近平台物理极限的轨迹是可行的。

因此, NMPC 在利用自主无人机的全部能力方面表现出色, 能够以更安全、更高效的方式操作, 这是由于它在处理参考轨迹不可行性方面的优势。未来的工作可能会进一步将 NMPC 与微分平坦性特性结合, 以在保持其处理动态不可行轨迹优势的同时降低 NMPC 的计算成本。NMPC 未能收敛的原因也需要进一步研究, 以应对更多的模型不确定性。

九、结论

本研究系统地比较了 NMPC 和 DFBC 两种用于跟踪敏捷轨迹的最先进四旋翼控制器。通过大量仿真和实际实验, 评估了两种方法在精度、鲁棒性和计算效率方面的性能。根据结果, 我们报告了两种方法的优缺点。本研究还评估了 INDI 内环控制对两种方法的影响。实际飞行结果显示, 在最高可达 20 m/s 的速度下, 应用 INDI 内环控制对 NMPC 和 DFBC 方法都是必要的。高速气动阻力模型的影响也被

经过评估, 发现其影响力比内部回路控制器小。

参考文献

- [1] G. Loianno 和 D. Scaramuzza, “基于视觉的无人机导航未来挑战与机遇特刊,” *Journal of Field Robotics*, 第37卷, 第4期, 页495–496, 2020.
- [2] S. Rajendran 和 S. Srinivas, “城市出行的空中出租车服务: 近期发展、未来挑战与机遇的批判性回顾,” *Transportation research part E: logistics and transportation review*, 第143卷, 页102090, 2020.
- [3] D. Bicego, J. Mazzetto, R. Carli, M. Farina, 和 A. Franchi, “具有增强执行器模型的多旋翼无人机通用设计非线性模型预测控制,” *Journal of Intelligent & Robotic Systems*, 第100卷, 第3期, 页1213–1247, 2020.
- [4] P. Foehn, A. Romero, 和 D. Scaramuzza, “四旋翼航点飞行的时间最优规划,” *Science Robotics*, 第6卷, 第56期, 页eabh1221, 2021.
- [5] G. Torrente, E. Kaufmann, P. Föhn, 和 D. Scaramuzza, “用于四旋翼的数据驱动模型预测控制,” *IEEE Robotics and Automation Letters*, 第6卷, 第2期, 页3769–3776, 2021.
- [6] M. Kamel, K. Alexis, M. Achtelik, 和 R. Siegwart, “用于多旋翼姿态的快速非线性模型预测控制”
- [10] H. Nguyen, M. Kamel, K. Alexis, 和 R. Siegwart, “微型飞行器的模型预测控制: 综述,” 收录于 *2021 European Control Conference (ECC)*, 第 1556–1563 页, IEEE, 2021.
- [11] A. Romero, P. Foehn, 和 D. Scaramuzza, “用于时间最优四旋翼飞行的模型预测轮廓控制,” *IEEE Transactions on Robotics*, 2022.
- [12] M. Faessler, A. Franchi, 和 D. Scaramuzza, “受转子阻力影响的四旋翼动力学的微分平坦性以实现高速轨迹的精确跟踪,” *IEEE Robotics and Automation Letters*, 卷. 3, 期. 2, 页. 620–626, 2017.
- [13] E. Tal 和 S. Karaman, “使用增量非线性动态反演和微分平坦性精确跟踪激进四旋翼的轨迹,” *IEEE Trans-*

- actions on Control Systems Technology*, vol. 29, no. 3, pp. 1203–1218, 2020.
- [14] T. A. Johansen and T. I. Fossen, “Control allocation—a survey,” *Automatica*, vol. 49, no. 5, pp. 1087–1103, 2013.
 - [15] E. J. Smeur, Q. Chu, and G. C. de Croon, “Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles,” *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 3, pp. 450–461, 2016.
 - [16] S. Sun, X. Wang, Q. Chu, and C. de Visser, “Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors,” *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 116–130, 2020.
 - [17] T. P. Nascimento and M. Saska, “Position and attitude control of multi-rotor aerial vehicles: A survey,” *Annual Reviews in Control*, vol. 48, pp. 129–146, 2019.
 - [18] H. Lee and H. J. Kim, “Trajectory tracking control of multirotors from modelling to experiments: A survey,” *International Journal of Control, Automation and Systems*, vol. 15, no. 1, pp. 281–292, 2017.
 - [19] S. Khatoun, D. Gupta, and L. Das, “Pid & lqr control for a quadrotor: Modeling and simulation,” in *2014 international conference on advances in computing, communications and informatics (ICACCI)*, pp. 796–802, IEEE, 2014.
 - [20] W. Dong, G.-Y. Gu, X. Zhu, and H. Ding, “Modeling and control of a quadrotor uav with aerodynamic concepts,” in *Proceedings of World Academy of Science, Engineering and Technology*, no. 77, p. 437, World Academy of Science, Engineering and Technology (WASET), 2013.
 - [21] H. Voos, “Nonlinear control of a quadrotor micro-uav using feedback-linearization,” in *2009 IEEE International Conference on Mechatronics*, pp. 1–6, IEEE, 2009.
 - [22] T. Madani and A. Benallegue, “Backstepping control for a quadrotor helicopter,” in *2006 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3255–3260, IEEE, 2006.
 - [23] E. Fresk and G. Nikolakopoulos, “Full quaternion based attitude control for a quadrotor,” in *2013 European control conference (ECC)*, pp. 3864–3869, IEEE, 2013.
 - [24] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor uav on $se(3)$,” in *49th IEEE conference on decision and control (CDC)*, pp. 5420–5425, IEEE, 2010.
 - [25] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *2011 IEEE international conference on robotics and automation*, pp. 2520–2525, IEEE, 2011.
 - [26] R. Mahony, V. Kumar, and P. Corke, “Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor,” *IEEE Robotics and Automation magazine*, vol. 19, no. 3, pp. 20–32, 2012.
 - [27] S. Sun, C. C. de Visser, and Q. Chu, “Quadrotor gray-box model identification from high-speed flight data,” *Journal of Aircraft*, vol. 56, no. 2, pp. 645–661, 2019.
 - [28] H. Huang, G. M. Hoffmann, S. L. Waslander, and C. J. Tomlin, “Aerodynamics and control of autonomous quadrotor helicopters in aggressive maneuvering,” in *2009 IEEE international conference on robotics and automation*, pp. 3277–3282, IEEE, 2009.
 - [29] M. Faessler, D. Falanga, and D. Scaramuzza, “Thrust mixing, saturation, and body-rate control for accurate aggressive quadrotor flight,” *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 476–482, 2016.
 - [30] E. Smeur, D. Höppener, and C. De Wagter, “Prioritized control allocation for quadrotors subject to saturation,” in *International Micro Air Vehicle Conference and Flight Competition*, no. September, pp. 37–43, 2017.
 - [31] D. Brescianini and R. D’Andrea, “Tilt-prioritized quadrotor attitude control,” *IEEE Transactions on Control Systems Technology*, vol. 28, no. 2, pp. 376–387, 2018.
 - [32] H. Zaki, M. Unel, and Y. Yildiz, “Trajectory control of a quadrotor using a control allocation approach,” in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 533–539, IEEE, 2017.
 - [33] M. Kamel, M. Burri, and R. Siegwart, “Linear vs nonlinear mpc for trajectory tracking applied to rotary wing micro aerial vehicles,” *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 3463–3469, 2017.
 - [34] M. Bangura and R. Mahony, “Real-time model predictive control for quadrotors,” *IFAC Proceedings Volumes*, vol. 47, no. 3, pp. 11773–11780, 2014.
 - [35] K. Alexis, G. Nikolakopoulos, and A. Tzes, “On trajectory tracking model predictive control of an unmanned quadrotor helicopter subject to aerodynamic disturbances,” *Asian Journal of Control*, vol. 16, no. 1, pp. 209–224, 2014.
 - [36] B. Houska, H. J. Ferreau, and M. Diehl, “Acado toolkit—an open-source framework for automatic control and dynamic optimization,” *Optimal Control Applications and Methods*, vol. 32, no. 3, pp. 298–312, 2011.
 - [37] Y. Chen, M. Bruschetta, E. Picotti, and A. Beghi, “Matmpc—a matlab based toolbox for real-time nonlinear model predictive control,” in *2019 18th European Control Conference (ECC)*, pp. 3365–3370, IEEE, 2019.
 - [38] J. A. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “Casadi: a software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
 - [39] M. Jacquet and A. Franchi, “Motor and perception constrained nmppc for torque-controlled generic aerial vehicles,” *IEEE Robotics and Automation Letters*, 2020.
 - [40] M. Diehl, H. G. Bock, H. Diedam, and P.-B. Wieber, “Fast direct multiple shooting algorithms for optimal robot control,” in *Fast motions in biomechanics and robotics*, pp. 65–93, Springer, 2006.
 - [41] R. Verschuere, G. Frison, D. Kouzoupis, N. van Duijckeren, A. Zanelli, R. Quirynen, and M. Diehl, “Towards a modular software package for embedded optimization,” *IFAC-PapersOnLine*, vol. 51, no. 20, pp. 374–380, 2018.
 - [42] H. J. Ferreau, C. Kirches, A. Potschka, H. G. Bock, and M. Diehl, “qpOASES: A parametric active-set algorithm for quadratic programming,” *Mathematical Programming Computation*, vol. 6, no. 4, pp. 327–363, 2014.
 - [43] X. Wang, E.-J. Van Kampen, Q. Chu, and P. Lu, “Stability analysis for incremental nonlinear dynamic inversion control,” *Journal of Guidance, Control, and Dynamics*,

- actions on Control Systems Technology, 第29卷, 第3期, 页码1203–1218, 2020年。[14] T. A. Johansen 和 T. I. Fossen, “控制分配——综述”, *Automatica*, 第49卷, 第5期, 页码1087–1103, 2013年。[15] E. J. Smeur, Q. Chu 和 G. C. de Croon, “微型无人机姿态控制的自适应增量非线性动态反演”, *Journal of Guidance, Control, and Dynamics*, 第39卷, 第3期, 页码450–461, 2016年。[16] S. Sun, X. Wang, Q. Chu 和 C. de Visser, “四旋翼无人机在两个对向旋翼完全失效情况下的增量非线性容错控制”, *IEEE Transactions on Robotics*, 第37卷, 第1期, 页码116–130, 2020年。[17] T. P. Nascimento 和 M. Saska, “多旋翼飞行器的位置与姿态控制: 综述”, *Annual Reviews in Control*, 第48卷, 页码129–146, 2019年。[18] H. Lee 和 H. J. Kim, “多旋翼无人机轨迹跟踪控制: 从建模到实验的综述”, *International Journal of Control, Automation and Systems*, 第15卷, 第1期, 页码281–292, 2017年。[19] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[20] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[21] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[22] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[23] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[24] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[25] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[26] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[27] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[28] S. Khatoun, D. Gupta 和 L. Das, “四旋翼无人机的PID和LQR控制: 建模与仿真”, 收录于 2014 international conference on advances in computing, communications and informatics (ICACCI), 页码796–802, IEEE, 2014年。[29] M. Faessler, D. Falanga, 和 D. Scaramuzza, “用于精确激进四旋翼飞行的推力混合、饱和和机体角速度控制”, *IEEE Robotics and Automation Letters*, 第2卷, 第2期, 第476–482页, 2016年。[30] E. Smeur, D. Höppener, 和 C. De Wagter, “受饱和限制的四旋翼优先控制分配”, 载于 *International Micro Air Vehicle Conference and Flight Competition*, 第九月期, 第37–43页, 2017年。[31] D. Brescianini 和 R. D’Andrea, “倾斜优先四旋翼姿态控制”, *IEEE Transactions on Control Systems Technology*, 第28卷, 第2期, 第376–387页, 2018年。[32] H. Zaki, M. Unel, 和 Y. Yildiz, “利用控制分配方法对四旋翼进行轨迹控制”, 载于 2017 International Conference on Unmanned Aircraft Systems (ICUAS), 第533–539页, IEEE, 2017年。[33] M. Kamel, M. Burri, 和 R. Siegwart, “应用于旋翼微型无人机的轨迹跟踪的线性与非线性MPC”, *IFAC-PapersOnLine*, 第50卷, 第1期, 第3463–3469页, 2017年。[34] M. Bangura 和 R. Mahony, “四旋翼的实时模型预测控制”, *IFAC Proceedings Volumes*, 第47卷, 第3期, 第11773–11780页, 2014年。[35]

vol. 42, no. 5, pp. 1116–1129, 2019.

- [44] W. Giernacki, M. Skwierczyński, W. Witwicki, P. Wroński, and P. Kozierski, “Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering,” in *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*, pp. 37–42, IEEE, 2017.

第42卷, 第5期, 页1116–1129, 2019。[44] W. Giernacki, M. Skwierczyński, W. Witwicki, P. Wroński, 和 P. Koziński, “Crazyflie 2.0 四旋翼作为机器人学与控制工程研究与教育的平台”, 发表于 2017 22nd *International Conference on Methods and Models in Automation and Robotics (MMAR)*, 页37–42, IEEE, 2017。